



المعهد العالي للإعلامية وتكنولوجيات الاتصال بسوسة

End-of-Studies Internship Report

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in
Computer Science

Specialization: Computer Science and Multimedia

Development of a Modern Invoicing Solution Based on Artificial Intelligence

Done by :

Dhia eddine Louati

Academic supervisor: Mr. Wissem Fathallah

Professional supervisor: Mr. Housseem Eddine Chibouni

Host company



Academic year: 2025/2026

Higher Institute of Computer Science and Communication Technologies of Sousse (ISITCom)

Tél/Fax : +216 73 37 15 71 / +216 73 36 44 11

To my family, whose unconditional love, endless patience, and unwavering support have been my anchor through every storm and my wings in every success. This work is as much yours as it is mine.

To my sister, for her constant presence, her words of encouragement when I doubted myself, and for believing in me even when I did not.

To my academic supervisor, Mr. Wissem Fathallah, for his invaluable guidance, scientific rigor, and the trust he placed in me. His advice and patience shaped this work and helped me grow as a researcher.

To all my friends and colleagues, who shared moments of joy and difficulty, who listened, advised, and never let me give up.

To all those who believe in me, who see light where there is shadow, and who trust that effort always bears fruit.

This dedication is a humble tribute to each of you.

Acknowledgements

First and foremost, I would like to thank Allah, the Almighty, for giving me the strength, courage, and patience necessary to successfully complete this final year project.

I would like to express my deep gratitude to my academic supervisor, Mr. Wissem Fathallah, for his availability, valuable advice, scientific rigor, and relevant guidance, which enabled me to structure my thinking and overcome the technical difficulties encountered throughout this project.

My thanks also go to my professional supervisor, Mr. Housseem Eddine Chibouni, project manager at Zedney Creative, for his warm welcome within the company, for the trust he placed in me, and for sharing his professional expertise. His constant support was essential to the success of this work.

Finally, I would like to thank all the teachers of ISITCOM for the quality of the education they provided me during my three years of study, as well as the members of the jury for their interest in my work.

Abstract

As part of optimizing corporate financial processes, this final year project aims to develop a comprehensive web-based invoicing platform integrating an intelligent assistant. The application, built with NestJS (backend), Next.js (frontend) and MySQL (database), enables centralized management of quotes, invoices, payments, customers and products. Particular emphasis is placed on security through JWT authentication and role-based access control (RBAC). The platform also offers an innovative AI assistant capable of analyzing invoices, predicting payment delays, detecting anomalies, generating summaries and interacting via a chatbot. The Scrum agile methodology was adopted to ensure iterative and quality delivery. This report details the entire development cycle, from requirements analysis to implementation, including testing and DevOps deployment.

Keywords: Invoicing, NestJS, Next.js, TypeScript, JWT, RBAC, AI Assistant, LLM, RAG, Docker, Scrum.

Résumé

Dans le cadre de l'optimisation des processus financiers en entreprise, ce projet de fin d'études vise le développement d'une plateforme web complète de facturation intégrant un assistant intelligent. L'application, construite avec NestJS (backend), Next.js (frontend) et MySQL (base de données), permet la gestion centralisée des devis, factures, paiements, clients et articles. Un accent particulier est mis sur la sécurité grâce à l'authentification JWT et un contrôle d'accès basé sur les rôles (RBAC). La plateforme offre également un assistant IA innovant capable d'analyser les factures, de prédire les retards de paiement, de détecter des anomalies, de générer des résumés et d'interagir via un chatbot. La méthodologie agile Scrum a été adoptée pour assurer une livraison itérative et de qualité. Ce rapport détaille l'ensemble du cycle de développement, de l'analyse des besoins à l'implémentation, en passant par les tests et le déploiement DevOps.

Mots-clés : Facturation, NestJS, Next.js, TypeScript, JWT, RBAC, Assistant IA, LLM, RAG, Docker, Scrum.

Contents

Acknowledgements	2
Abstract	3
Résumé	4
General Introduction	8
1 Project Context and Analysis of Existing Systems	10
1.1 Introduction	10
1.2 Presentation of the Host Organisation	10
1.2.1 Internal Organisation and Missions	10
1.2.2 Project Context at Zedney Creative	11
1.3 Study and Critique of Existing Systems	11
1.3.1 Observed Current Practices	11
1.3.2 Limitations and Shortcomings	12
1.3.3 Benchmark of Software Solutions	13
1.4 Proposed Solution	13
1.4.1 Functional Objectives	13
1.4.2 Technical Objectives	14
1.4.3 Added Value Compared to Existing Systems	14
1.5 Working Methodology: Scrum	14
1.5.1 Principles and Roles	14
1.5.2 Scrum Artifacts	15
1.5.3 Scrum Ceremonies	15
1.5.4 Sprint Organisation	15
1.6 Development Environment	17
1.6.1 Hardware Configuration	17
1.6.2 Software and Tools	17
1.6.3 Detailed Technologies	17
1.7 Overall Software Architecture	20
1.7.1 General Architecture	20
1.7.2 Detailed Backend Architecture (NestJS)	20
1.7.3 Frontend Architecture (Next.js)	20
1.7.4 Architecture Diagram	21
1.8 Chapter Conclusion	21
2 Requirements Analysis and Planning	22
2.1 Introduction	22
2.2 Stakeholder Identification	22
2.2.1 Super Administrator	22
2.2.2 Accountant / Financial Manager	22

2.2.3	Sales Representative / Account Manager	23
2.3	Requirements Analysis (User Stories)	23
2.3.1	Theme: Authentication and Access Management	23
2.3.2	Theme: Core Data Management	24
2.3.3	Theme: Quotes and Invoicing	24
2.3.4	Theme: AI Assistant	24
2.3.5	Theme: Security and Audit	25
2.3.6	Theme: Reports and Export	25
2.4	Sprint Planning and Backlog	25
2.4.1	Sprint 1: Foundation and Security (3 weeks)	25
2.4.2	Sprint 2: Core Data (3 weeks)	25
2.4.3	Sprint 3: Invoicing Cycle, AI and PDF (3 weeks)	26
2.5	Chapter Conclusion	27
3	System Architecture and Design	28
3.1	Introduction	28
3.2	Use Case Diagrams	28
3.2.1	Textual Description of a Use Case: "Convert Quote to Invoice"	29
3.3	Class Diagram	30
3.4	Sequence Diagrams	31
3.4.1	JWT Authentication	32
3.4.2	Creating an Invoice from a Quote	32
3.5	Activity Diagram	33
3.6	Component Diagram	33
3.7	Deployment Diagram	34
3.8	Database Modelling (ERD)	34
3.9	Compliance with Architecture Principles	35
3.10	Chapter Conclusion	36
4	Backend Development with NestJS	37
4.1	Introduction	37
4.2	Project Structure	37
4.3	Setting up Authentication (JWT)	38
4.3.1	JWT Strategy	38
4.3.2	Authentication Service	38
4.3.3	Authentication Controller	39
4.4	Role and Permission Management (RBAC)	39
4.4.1	Roles Decorator	39
4.4.2	RolesGuard	40
4.4.3	Usage in a Controller	40
4.5	Invoicing Module	40
4.6	Data Validation (DTOs and Pipes)	41
4.7	Error Handling and Logging	42
4.8	API Documentation with Swagger	43
4.9	Chapter Conclusion	43

5	Frontend Development with Next.js	44
5.1	Introduction	44
5.2	Next.js Project Structure	44
5.3	Frontend Authentication Management	45
5.4	API Integration with Axios	46
5.4.1	Invoice Service (Client-side)	47
5.5	Key Pages (App Router)	47
5.5.1	Login Page (Client Component)	47
5.5.2	Dashboard Page (Client Component with Data Fetching)	48
5.5.3	Quote Creation Page (React Hook Form + useFieldArray)	49
5.6	PDF Generation (Client-side)	50
5.7	Server Components for Performance	51
5.8	Responsive Design with Tailwind CSS	52
5.9	Chapter Conclusion	53
6	Authentication and Security	54
6.1	Introduction	54
6.2	JWT Authentication	54
6.2.1	Token Refresh	54
6.3	Password Hashing (bcrypt)	54
6.4	Role-Based Access Control (RBAC)	55
6.4.1	Example of Admin-Only Route	55
6.5	Protection Against Attacks	55
6.5.1	SQL Injection	55
6.5.2	Cross-Site Scripting (XSS)	55
6.5.3	Cross-Site Request Forgery (CSRF)	55
6.5.4	Input Validation (DTOs)	55
6.5.5	Rate Limiting	55
6.5.6	CORS	56
6.6	AI Assistant Security	56
6.7	Chapter Conclusion	57
7	Invoice Lifecycle and Business Logic	58
7.1	Introduction	58
7.2	Quote Management	58
7.2.1	Total Calculations	58
7.3	Quote → Invoice Conversion	59
7.3.1	Sequence Manager (SequenceService)	59
7.4	Payment Tracking and Balance	60
7.5	PDF Generation	60
7.6	Activity Logs (Audit)	61
7.7	Chapter Conclusion	62
8	AI Assistant and Intelligent Features	63
8.1	Introduction	63
8.2	Overall AI Assistant Architecture	63
8.3	Detailed Features	64
8.3.1	Invoice Analysis and Automatic Summarisation	64
8.3.2	Payment Delay Prediction	64

8.3.3	Anomaly Detection	64
8.3.4	Conversational Chatbot	65
8.3.5	Natural Language Search	65
8.3.6	Item Recommendation	65
8.3.7	Intelligent Reminder Sending	65
8.4	Technical Implementation	65
8.4.1	Integration with OpenAI API	65
8.4.2	RAG Pipeline with pgvector	66
8.4.3	Prompt Engineering for the Assistant	66
8.4.4	Delay Prediction (scikit-learn model)	67
8.4.5	Performance Optimisation and Caching	67
8.5	AI Security	67
8.6	Chapter Conclusion	68
9	Database Design and Optimisation	69
9.1	Introduction	69
9.2	Conceptual (ERD) and Logical (LDM) Models	69
9.3	Relational Schema (SQL) – Main Tables	69
9.3.1	Table users	69
9.3.2	Table roles	70
9.3.3	Table firms	70
9.3.4	Table invoices	70
9.3.5	Table invoice_lines	71
9.3.6	Table payments	71
9.3.7	Table sequences	71
9.3.8	Table logs	72
9.3.9	Table ai_requests	72
9.4	Normalisation and Referential Integrity	72
9.5	Indexing and Performance Optimisation	72
9.6	Transactions and Locking	73
9.7	Backup and Recovery	73
9.8	Chapter Conclusion	75
10	Testing and Validation	76
10.1	Introduction	76
10.2	Testing Strategy	76
10.3	Backend Unit Tests (Jest)	76
10.4	API Integration Tests (Supertest)	77
10.5	Frontend Tests (Jest + React Testing Library)	77
10.6	Security Tests	78
10.7	Performance Tests	78
10.8	AI Assistant Tests	78
10.9	Overall Results	78
10.10	Chapter Conclusion	79

11 Deployment and DevOps	80
11.1 Introduction	80
11.2 Containerisation with Docker	80
11.2.1 Dockerfile for the NestJS Backend	80
11.2.2 Dockerfile for the Next.js Frontend (Production Multi-Stage)	80
11.2.3 Docker Compose for the Complete Environment	81
11.3 Continuous Integration with GitHub Actions	82
11.4 Deployment on a Cloud Server (DigitalOcean)	83
11.4.1 Nginx Configuration for HTTPS	83
11.5 Monitoring and Logging	84
11.6 Backup and Disaster Recovery	84
11.7 Chapter Conclusion	85
General Conclusion	86
11.8 Summary of Achievements	86
11.9 Notable Technical Points	86
11.10 Difficulties Encountered	86
11.11 Future Improvements	87
11.12 Personal and Professional Benefits	87
A Appendix A: Complete API Routes	89
B Appendix B: Environment Variables	90
C Appendix C: Example Prompts for the AI Assistant	91
C.1 Query Classification Prompt	91
C.2 Payment Reminder Generation Prompt	91
D Appendix D: Docker Configuration and Scripts	92
D.1 Full Deployment Script (deploy.sh)	92
D.2 Database Migration Script	92
E Appendix E: Additional Screenshots	93
F Appendix F: Quick Installation Guide	95
F.1 Prerequisites	95
F.2 Installation	95

List of Figures

1.1	Typical manual invoicing process (illustrative diagram)	12
1.2	Scrum process	16
1.3	Complete technology stack	19
1.4	Overall architecture of the ZC - invoice platform	21
2.1	User story distribution per sprint	26
3.1	Overall use case diagram	29
3.2	Simplified class diagram	30
3.3	Sequence diagram of the authentication process	32
3.4	Sequence diagram of quote → invoice conversion	32
3.5	Activity diagram of the document lifecycle	33
3.6	Component diagram (backend)	34
3.7	Deployment diagram	34
3.8	Entity-Relationship Diagram (ERD)	35
5.1	Login page	52
5.2	dashboard page	53
5.3	Invoice list with filters	53
6.1	Security logs consultation (failed attempts, sensitive actions)	56
7.1	Invoice detail with PDF option and payment history	62
8.1	AI Assistant Architecture	63
8.2	AI chatbot interface	68
8.3	AI insights dashboard (anomalies, predictions)	68
10.1	Response time under load (k6 test)	78
11.1	CI/CD pipeline and deployment architecture	84
E.1	User management interface (admin)	93
E.2	Item Catalogue	93
E.3	Payment tracking dashboard	94

List of Tables

2.1	Role-based access matrix (excerpt)	23
3.1	Description of the main classes	31
9.1	Complete relational schema of the invoicing database	73
10.1	Performed security tests	78
A.1	Main API routes	89

General Introduction

In a constantly evolving economic environment, the digital transformation of administrative and financial processes has become an essential strategic lever for companies of all sizes. Invoicing, which lies at the core of commercial relationships, is no longer limited to the simple issuance of a document summarizing services. It now covers the entire sales cycle: quotations, invoice generation, payment tracking, reminders, financial reporting, and customer management. Faced with this increasing complexity, many small and medium-sized enterprises struggle to adopt efficient, secure, and scalable tools, often relying on inadequate office solutions that are prone to errors.

It is within this context that this final year project, entitled Development of an Intelligent Invoicing Platform Integrated with an AI Assistant, is carried out. The main objective is to design, develop, and deploy a centralized, robust, and intelligent web application capable of automating the entire invoicing workflow while providing advanced decision-support features based on artificial intelligence. This project was developed independently by the author in response to a real need expressed by the host company, Zedney Creative, aiming to modernize its internal financial management tools and deliver a more innovative solution.

The development of this platform is based on a modern and proven technology stack: **NestJS** for the backend, leveraging the power of Node.js and the expressiveness of TypeScript; **Next.js** for the frontend, providing a fast, responsive, and seamless user experience; and **MySQL** for data persistence, ensuring integrity and transactional consistency. Authentication is handled using JWT tokens combined with a role-based access control system (RBAC). In addition, the integration of an **intelligent assistant** (AI) — capable of analyzing invoices, predicting payment delays, detecting anomalies, generating automatic summaries, and interacting with users via a chatbot — represents the main innovation of this project.

The project follows the **Scrum** methodology, enabling iterative and incremental development, rapid adaptation to changing requirements, and continuous delivery of value-added features. The development was organized into three sprints covering respectively: (1) secure system foundation and user management, (2) core data management (customers, items, taxes), and (3) the complete invoicing cycle (quotes, invoices, payments, PDF generation) along with the implementation of AI modules.

This report presents all the work carried out and is structured into five main chapters:

- **Chapter 1: Context and analysis of the existing system** – Presentation of the host organization, critical study of existing solutions, justification of the proposed solution, Scrum methodology, and technical environment.
- **Chapter 2: Requirements analysis and planning** – Identification of stakeholders, definition of user stories, product backlog creation, and sprint planning.
- **Chapter 3: System architecture and design** – Description of the overall software architecture, UML diagrams (use case, class, sequence, deployment), and data modeling.

- **Chapters 4 to 11** – Backend and frontend development, security, invoicing workflow, AI assistant, database design, testing, and DevOps deployment.
- **General Conclusion** – Evaluation of achievements, assessment of objectives, challenges encountered, and future improvements.

This report is written and developed solely by the author and is intended for academic evaluators, industry professionals, and anyone interested in the design of financial information systems integrating artificial intelligence. It is hoped that this work will serve as a useful reference for future similar projects.

Chapter 1

Project Context and Analysis of Existing Systems

1.1 Introduction

This first chapter lays the essential groundwork for understanding the project. We describe the professional environment in which our work takes place, through a presentation of the host organisation and its field of activity. We then conduct a critical study of existing invoicing solutions – whether manual or software-based – in order to identify their strengths and weaknesses. Based on this analysis, we justify the choice of developing a custom solution that integrates advanced security and artificial intelligence features. Finally, we present the agile Scrum methodology we adopted, as well as the technical environment and technologies used to carry out the project.

1.2 Presentation of the Host Organisation

The project was carried out within **Zedney Creative**, a Tunisian company specialising in consulting and developing innovative software solutions. Founded in 2018, Zedney Creative now has around twenty employees distributed across the following departments: web development, mobile applications, enterprise system integration, and cybersecurity. The company supports its clients (SMEs, large enterprises, public institutions) in their digital transformation by providing custom tools tailored to their business processes.

1.2.1 Internal Organisation and Missions

The technical team is structured into several agile cells:

- A *Backend* cell specialising in REST APIs, microservices and event-driven architectures (Node.js, NestJS, Spring Boot).
- A *Frontend* cell mastering React, Angular and Vue.js.
- A *DevOps* cell managing cloud infrastructure (AWS, Azure), containerisation (Docker, Kubernetes) and CI/CD pipelines.
- A *Data & AI* cell working on data analysis, machine learning and the integration of language models (LLMs).

The internship took place within the backend cell, with frequent interactions with the frontend and AI cells. Professional supervision was provided by Mr. Rafik Ben Salah, Senior Software Architect, who oversaw the entire invoicing project.

1.2.2 Project Context at Zedney Creative

Zedney Creative wanted to internalise an invoicing tool for its own administrative needs, but also to have a reusable platform for its clients. The existing system was limited to shared Excel files, scanned handwritten quotes and manual payment tracking. Management therefore launched an internal project to develop a “ready-to-use”, scalable solution with innovative features, notably an AI assistant for financial management assistance. This project was entrusted to the development team, and my internship was part of this initiative.

1.3 Study and Critique of Existing Systems

Before designing the new platform, an in-depth study of existing practices and tools was conducted.

1.3.1 Observed Current Practices

In Tunisian SMEs, several invoicing methods coexist:

1. **The purely manual method:** using word processors (Microsoft Word) to create quotes/invoices. Calculations are done with a calculator, and documents are printed or sent by email as attachments. Payment follow-up is kept in a notebook or spreadsheet.
2. **Advanced spreadsheets (Excel/Google Sheets):** some users employ workbooks with formulas and conditional formatting. Customer and item data are duplicated from one invoice to another. Traceability is limited, and the risk of formula errors is high.
3. **Standard accounting software (Sage, QuickBooks, Odoo):** these applications offer invoicing modules, but their price, complexity and functional overload make them unsuitable for very small structures. Moreover, they do not provide an integrated AI assistant for predictive analysis.

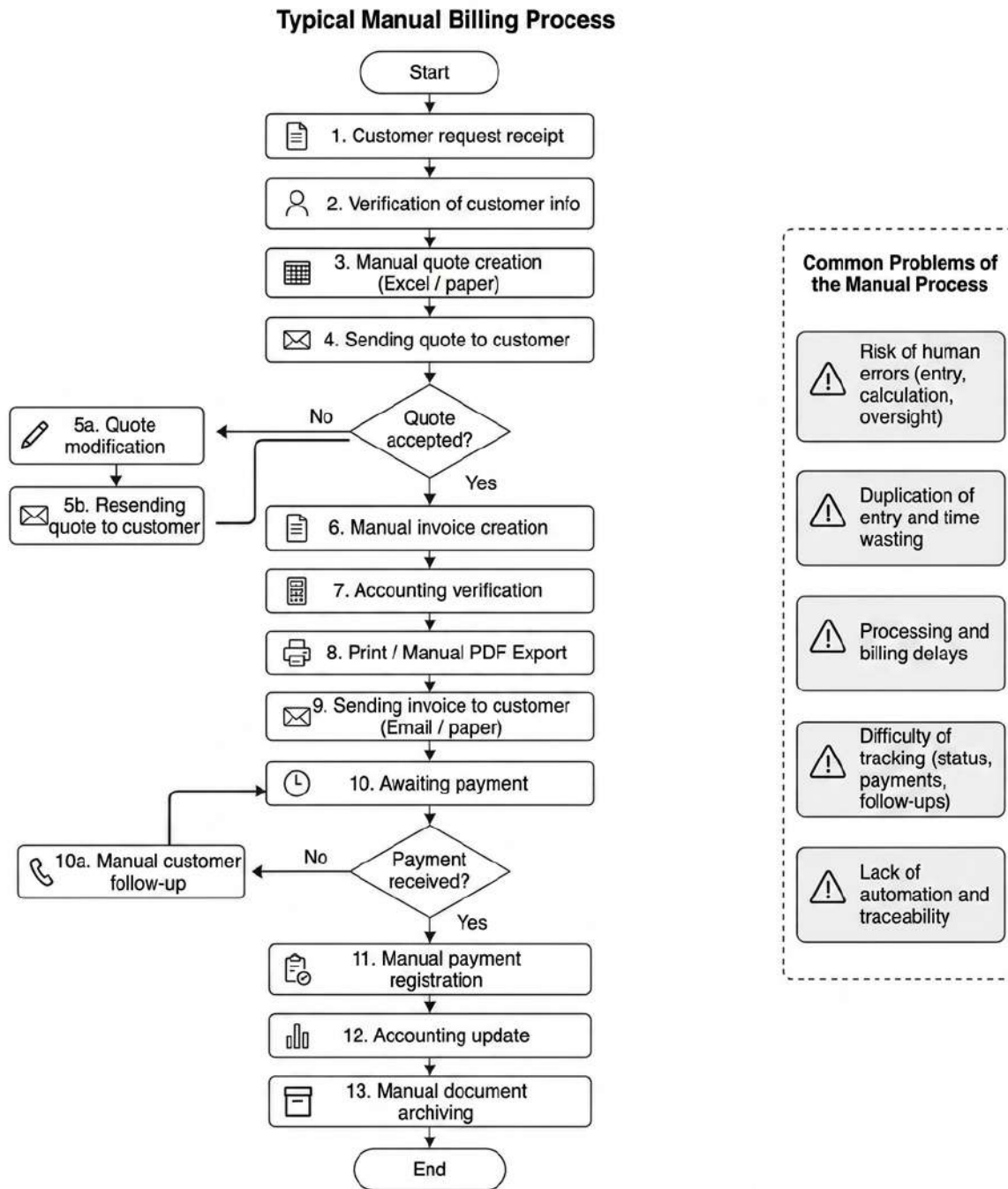


Figure 1.1: Typical manual invoicing process (illustrative diagram)

1.3.2 Limitations and Shortcomings

The critical analysis reveals several recurring problems:

- **Lack of centralisation:** customer information, items and documents are scattered across different workstations or shared folders.
- **High risk of errors:** manual calculation of pre-tax totals, VAT and tax-inclusive totals is a source of inconsistencies. An internal study showed that nearly 30% of manually issued invoices contained at least one calculation or VAT error.

- **Lack of traceability:** it is difficult to know who created, modified or deleted an invoice, and when. This poses an audit and compliance problem.
- **Inefficient payment reminders:** reminders are sent randomly, without systematic follow-up. The default rate increases.
- **Legal non-compliance:** invoice numbering is not always sequential or gap-free, which contradicts tax requirements (in particular Article 243 of the Tunisian Code of Obligations).
- **Lack of predictive analysis:** no existing solution provides forecasts of payment delays or automatic alerts based on customer history.

1.3.3 Benchmark of Software Solutions

A benchmark was carried out on three categories of products:

1. **Factur.io / Devis Facture Pro:** simple SaaS solutions, but limited in terms of role management, APIs and AI integration.
2. **Dolibarr / Odoo Community:** comprehensive open-source ERPs, but heavy to install and configure for a small SME. The interface is not very intuitive for basic use.
3. **Microsoft Dynamics 365 Business Central:** very powerful but extremely expensive and oversized for our target.

None of the evaluated solutions offer an integrated conversational AI assistant, nor payment delay prediction features. This therefore represents an innovation opportunity.

1.4 Proposed Solution

Based on these findings, we propose to develop an integrated invoicing platform, named **ZC - invoice**, addressing the following objectives:

1.4.1 Functional Objectives

- Centralise the management of clients (firms), contacts, items, taxes and financial documents.
- Automate the quote → invoice → payment cycle with configurable business rules (sequences, statuses).
- Ensure the generation of professional PDF documents, automatically numbered and compliant with legislation.
- Provide analytical dashboards (turnover, top clients, default rate).
- Integrate an AI assistant for: automatic invoice analysis, payment delay prediction, anomaly detection (duplicate invoice, abnormal amount), summary generation, an interactive chatbot, intelligent personalised reminder sending and optimisation recommendations.

1.4.2 Technical Objectives

- Modular backend architecture (NestJS) respecting SOLID principles and clean architecture.
- Responsive user interface, developed with Next.js and TypeScript for robustness.
- Documented REST API (Swagger/OpenAPI) secured with JWT + RBAC.
- Normalised relational database (3NF) with ACID transaction support.
- Containerisation with Docker and orchestration via Docker Compose to facilitate deployment.
- CI/CD pipeline (GitHub Actions) for continuous integration and deployment.
- AI assistant based on a local language model (Ollama) combined with RAG (Retrieval-Augmented Generation) using a vector database (pgvector) to ensure data confidentiality and operational independence.

1.4.3 Added Value Compared to Existing Systems

The solution stands out by:

- The native integration of a conversational and predictive AI assistant, directly accessible from the invoicing interface, operating entirely on local infrastructure.
- Advanced customisation of PDF templates and numbering rules.
- A comprehensive logging system for auditing all sensitive actions.
- A modern, extensible and maintainable architecture, facilitating the addition of new features (payment gateways, multi-currency, etc.).

1.5 Working Methodology: Scrum

The project development followed the agile Scrum methodology, adapted to innovative projects with evolving requirements.

1.5.1 Principles and Roles

- **Product Owner:** played by the professional supervisor Mr. Ben Salah, responsible for prioritising the product backlog.
- **Scrum Master:** played by the academic supervisor Mr. Karoui, facilitator of the process and guarantor of compliance with Scrum rules.
- **Development Team:** the intern student, in occasional collaboration with other company developers for code reviews.

1.5.2 Scrum Artifacts

- **Product Backlog:** an ordered list of all features expressed as user stories.
- **Sprint Backlog:** the selection of stories to be completed during a 3-week sprint.
- **Increment:** a functional version of the product at the end of each sprint.

1.5.3 Scrum Ceremonies

- **Sprint Planning:** meeting at the beginning of the sprint to define the goal and select stories.
- **Daily Stand-up:** 15-minute daily meeting to share progress and blockers.
- **Sprint Review:** presentation of the increment to the professional supervisor for validation.
- **Sprint Retrospective:** analysis of improvement points at the end of each sprint.

1.5.4 Sprint Organisation

The project was divided into three sprints of three weeks each:

1. **Sprint 1 (Weeks 1-3):** Setting up the foundation – JWT authentication, user and role management (RBAC), initial database.
2. **Sprint 2 (Weeks 4-6):** Core data management – CRUD for firms, contacts, items, taxes, sequences.
3. **Sprint 3 (Weeks 7-9):** Invoicing cycle – quotes, invoices, payments, PDF generation, logs, integration of the AI assistant (local LLM, RAG pipeline, predictive models).

This iterative breakdown allowed for early feedback and priority adjustments.

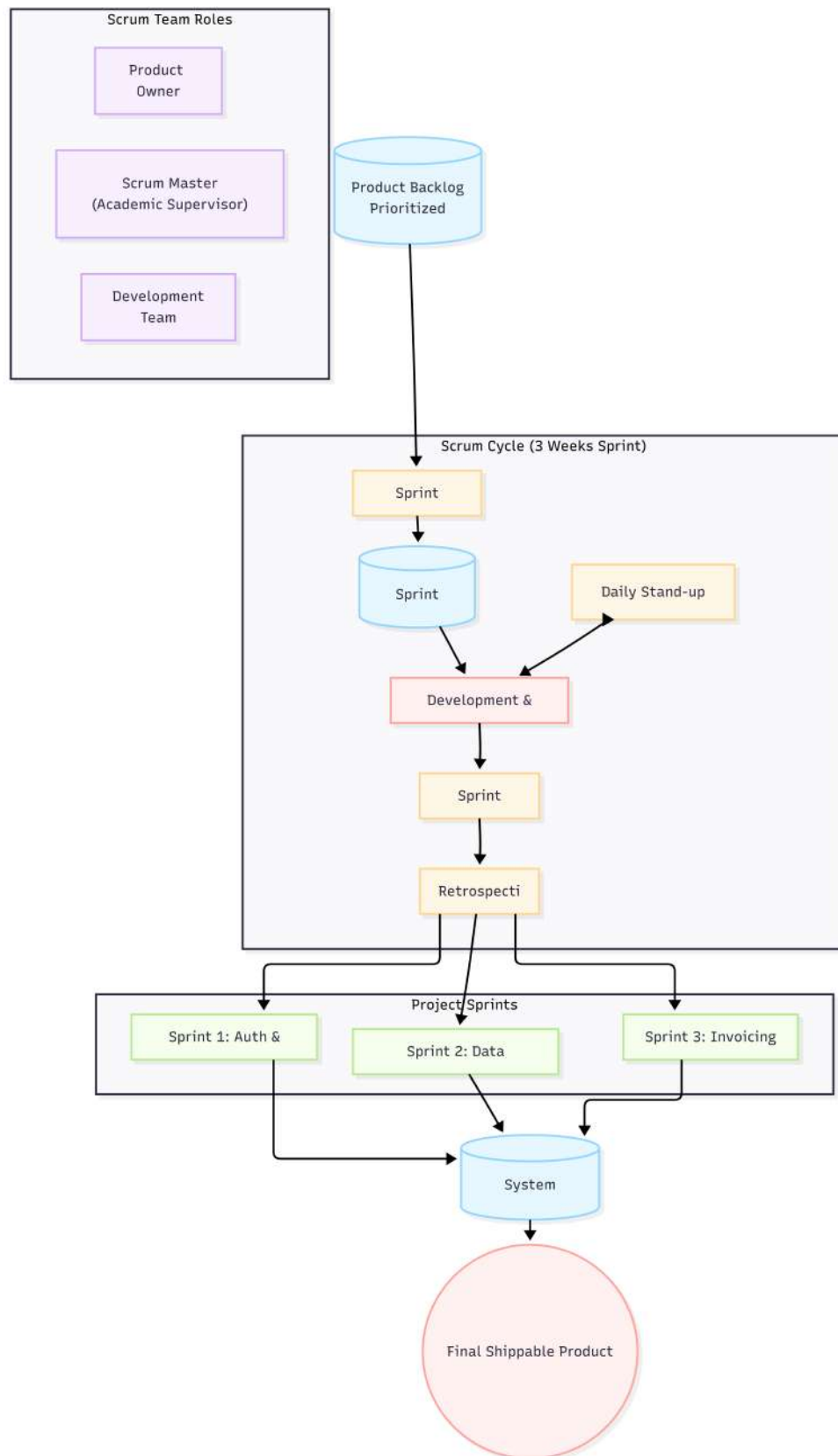


Figure 1.2: Scrum process

1.6 Development Environment

1.6.1 Hardware Configuration

- Laptop: Lenovo IdeaPad Gaming 3, AMD Ryzen 5 processor, NVIDIA GeForce RTX 3050 4GB GPU, 16 GB RAM, 512 GB SSD.
- Operating system: Windows 11 Professional + WSL2 (Ubuntu 22.04) for the Docker environment.

1.6.2 Software and Tools

- IDE: Visual Studio Code with extensions: TypeScript, ESLint, Prettier, NestJS snippets, Next.js snippets.
- Version control: Git (GitHub) – simplified Git Flow strategy (main, develop, feature/* branches).
- Database: MySQL 8.0 (instance via Docker) + MySQL Workbench for administration; PostgreSQL with pgvector for the vector database.
- API testing: Postman (shared collections) and Insomnia.
- Containerisation: Docker Desktop for Windows (WSL2 backend), Docker Compose for multi-container orchestration.
- CI/CD: GitHub Actions (workflows for tests and deployment to a staging server).
- Documentation: Swagger UI (integrated into NestJS via @nestjs/swagger).

1.6.3 Detailed Technologies

Backend: NestJS

NestJS is a progressive Node.js framework that uses TypeScript and draws inspiration from Angular's architecture (modules, controllers, services). It natively supports object-oriented, functional and reactive programming (RxJS). The advantages for our project:

- Modular structure facilitating the division of functionalities.
- Easy integration of ORMs (TypeORM, Sequelize) and validation libraries (class-validator).
- Guards, interceptors and pipes for security and data transformation.

NestJS was chosen for its robustness, scalability and large community.

Frontend: Next.js + TypeScript

Next.js is a React framework that provides server-side rendering (SSR), static site generation (SSG) and client-side rendering. It offers a complete structure for modern web applications. The combination with TypeScript provides static typing that reduces bugs and improves the development experience. We used:

- App Router for file-based routing and layout management.
- Server Components and Client Components to optimise performance.
- Axios for HTTP calls (or native fetch with Next.js extensions).
- Next.js API routes for lightweight backend endpoints (when needed).
- React Context API (with hooks) or Zustand for global state management (authentication, theme).
- React Hook Form for efficient form handling.
- Tailwind CSS for styling (chosen for its speed and consistency).

Next.js was selected over plain React for its performance benefits (SSR/SSG), built-in routing, and improved SEO capabilities – features that are valuable for a professional invoicing platform.

Database: MySQL and TypeORM

MySQL is an ACID-compliant relational DBMS, perfectly suited to the highly structured invoicing data. TypeORM, an ORM supported by NestJS, allows entities to be defined as TypeScript classes and facilitates migrations. It supports relationships (OneToMany, ManyToOne, etc.), transactions and advanced query builders.

Security: JWT, bcrypt, RBAC

Authentication is based on JSON Web Tokens (JWT), signed with a secret key. Passwords are hashed with bcrypt (cost factor 12). Role-based access control (RBAC) is implemented via a custom NestJS guard that verifies the role contained in the token before granting access to a route.

AI Assistant: Local Model (Ollama), RAG, pgvector

The AI assistant is implemented in three complementary components:

1. **Local language model:** we use a language model executed locally via Ollama. The selected model is Qwen2.5-7B-Instruct (quantised to 4-bit), adapted to a resource-constrained environment (16 GB RAM, GPU acceleration via RTX 3050). This local approach ensures data confidentiality, reduces dependency on external services, and guarantees better control over the system.

2. **RAG (Retrieval-Augmented Generation):** to enrich responses with user-specific data (invoices, customers, history), we integrated an RAG pipeline. Documents (invoices, notes, emails) are split into chunks, vectorised using a local embedding model (nomic-embed-text), and stored in a pgvector vector database (a PostgreSQL extension). In production, the user query is transformed into an embedding, then a similarity search is performed to retrieve the most relevant chunks. These chunks are subsequently injected into the prompt of the local model (Qwen2.5-7B) to generate a contextualised and reliable response.
3. **Specific functions:** the assistant also integrates advanced business features:
 - *Payment delay risk prediction:* a classification model (logistic regression) trained on historical data (average delay, amount, sector, seniority, payment behaviour, etc.) estimates the risk for each invoice. This model is exposed via a REST microservice.
 - *Anomaly detection:* business rules (abnormally high invoices, duplicates, inconsistencies) combined with unsupervised models such as Isolation Forest automatically identify unusual behaviours or documents.

ZC - invoice

Figure 1.3 – Complete Technology Stack

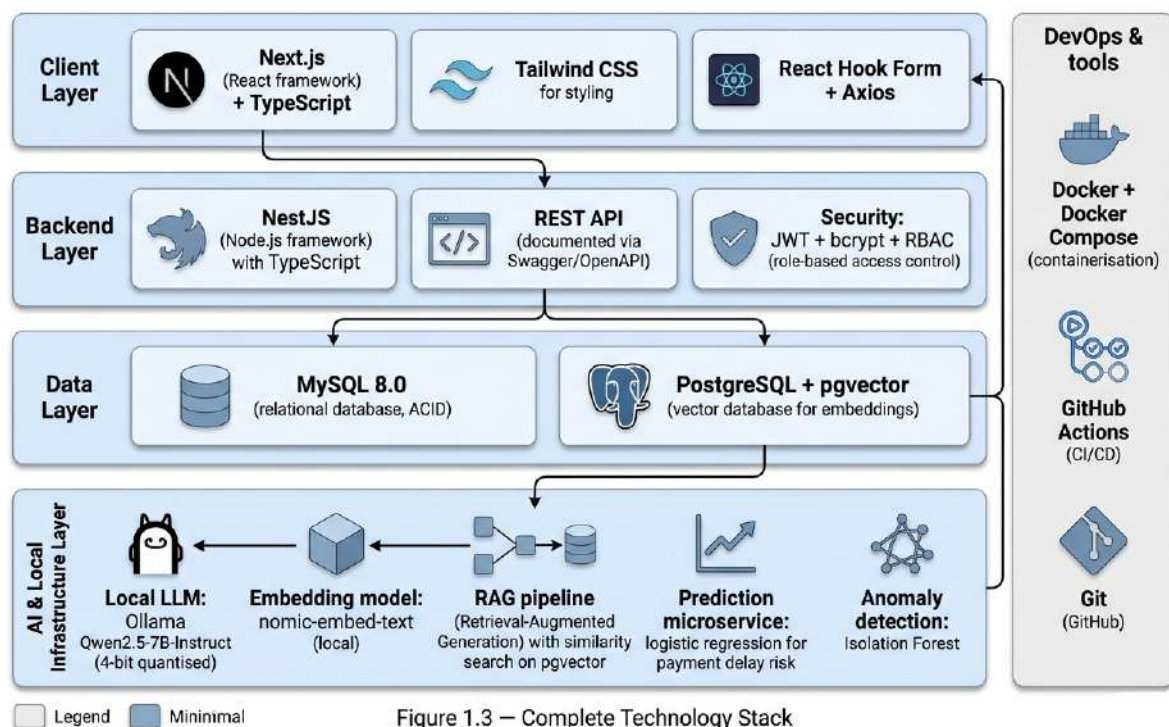


Figure 1.3: Complete technology stack

1.7 Overall Software Architecture

1.7.1 General Architecture

The platform architecture follows the **client-server** model with a clear separation between frontend and backend, communicating via a REST API. The frontend (Next.js application) can be served by a static web server or a Node.js server. The backend exposes endpoints protected by JWT and accesses the MySQL database via TypeORM. The AI assistant interacts with the local Ollama instance, the embedding model, and the pgvector vector database. Prediction and anomaly detection are provided by a lightweight Python microservice (or integrated into the NestJS backend via a bridge).

1.7.2 Detailed Backend Architecture (NestJS)

NestJS encourages a layered architecture:

- **Controllers:** responsible for handling HTTP requests (parameter validation, use of DTOs, sending appropriate responses). They delegate business logic to services.
- **Services:** contain business logic, calculations (VAT totals), transactions, communication with repositories. They are injectable (DI).
- **Repositories** (via TypeORM): each entity has a repository that allows CRUD operations and custom queries.
- **Modules:** group controllers, services and entities by application domain (AuthModule, UsersModule, InvoicesModule, AIModule, etc.).

1.7.3 Frontend Architecture (Next.js)

The frontend is structured according to the *feature-based* principle, leveraging the Next.js App Router:

- **Components:** reusable components (buttons, modals, forms) – typically client components.
- **App directory:** each route is defined by a folder, with `page.tsx` files representing pages (LoginPage, DashboardPage, InvoiceListPage, etc.).
- **Custom hooks:** abstraction of reusable logic (useAuth, useInvoices, useAI) – compatible with both server and client contexts.
- **Contexts / Stores:** global data providers (AuthContext, NotificationContext) using React Context or a lightweight state manager.
- **API service layer:** typed Axios (or fetch) functions for API calls (invoiceAPI, userAPI, aiAPI, etc.).
- **Server Components:** used for data fetching that does not require client-side interactivity, improving performance.

1.7.4 Architecture Diagram

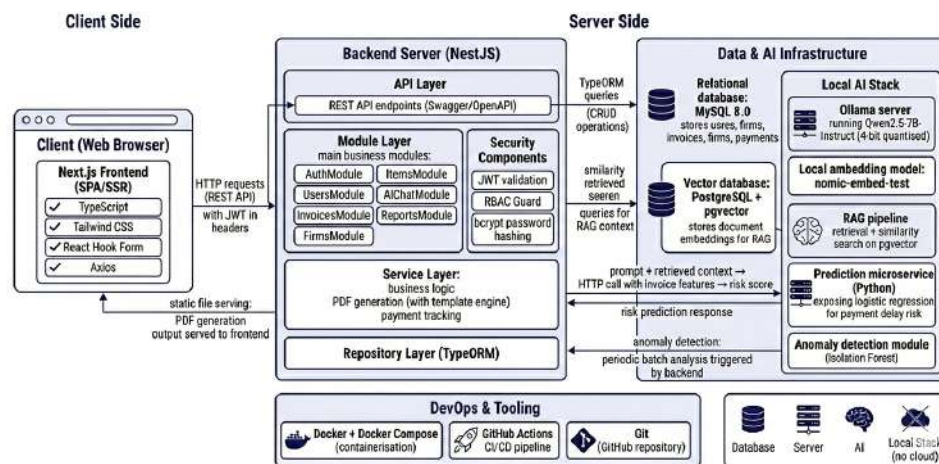


Figure 1.4: Overall architecture of the ZC - invoice platform

1.8 Chapter Conclusion

In this first chapter, we have outlined the project context by presenting the host organisation, the weaknesses of existing solutions and the opportunities for innovation, particularly around the local AI assistant (Ollama + RAG + pgvector). We have justified the choice of the Scrum methodology, the technical stack (NestJS, Next.js, MySQL, Ollama) and the overall architecture. This conceptual and technical foundation allows us to proceed confidently to the requirements analysis and planning, which will be the subject of the next chapter.

Chapter 2

Requirements Analysis and Planning

2.1 Introduction

Before any development begins, it is essential to understand precisely what users expect from the system. This chapter is devoted to identifying stakeholders, detailing requirements in the form of user stories, and planning sprints with the definition of the product backlog. This specification phase ensures that development will meet real needs and enable objective evaluation of the final product.

2.2 Stakeholder Identification

Analysis of the business context revealed three main types of stakeholders interacting with the platform.

2.2.1 Super Administrator

The administrator has full rights over the system. Their responsibilities include:

- Complete user management (creation, modification, deactivation, role assignment).
- Configuration of general parameters: taxes (VAT), numbering sequences, PDF templates.
- Consultation of audit logs to trace sensitive actions.
- Monitoring system performance and the AI assistant (request tracking, local resource usage).

2.2.2 Accountant / Financial Manager

The user responsible for daily financial management. They can:

- Create, modify and view client records (firms) and contacts.
- Manage the item catalogue and prices.
- Issue quotes, convert them into invoices, record payments, generate credit notes.
- Consult dashboards (turnover, outstanding payments) and export reports.
- Access the AI assistant for predictive analysis and optimisation suggestions.

2.2.3 Sales Representative / Account Manager

This role has more limited rights to preserve the integrity of financial data:

- Create and modify quotes (but cannot validate or convert them into invoices without accounting validation).
- View the list of clients and their contact information.
- View invoice history (without being able to modify or delete).
- Use the AI assistant to obtain customer account summaries and commercial recommendations.

Table 2.1: Role-based access matrix (excerpt)

Feature	Super Admin	Accountant	Sales Rep
User management	Yes	No	No
Tax configuration	Yes	No	No
Customer/item management	Yes	Yes	View only
Quote creation	Yes	Yes	Yes
Quote → invoice conversion	Yes	Yes	No
Payment recording	Yes	Yes	No
PDF generation	Yes	Yes	View only
Log consultation	Yes	No	No
AI assistant (analysis)	Yes	Yes	Limited

2.3 Requirements Analysis (User Stories)

Functional requirements were expressed in the form of user stories written in French (translated here). Each story follows the pattern: *As a [role], I want [action] so that [benefit]*. Below we present the most significant stories, organised by theme.

2.3.1 Theme: Authentication and Access Management

- **US-AUTH-01:** As a user, I want to log in with my email and password so that I can access the platform.
- **US-AUTH-02:** As an administrator, I want to create a user account and assign a role (admin, accountant, sales) so that I can control access rights.
- **US-AUTH-03:** As an administrator, I want to deactivate a user account so that I can revoke access in case of departure.
- **US-AUTH-04:** As a user, I want to change my password after authentication so that I can ensure the security of my account.

2.3.2 Theme: Core Data Management

- **US-CORE-01:** As an accountant, I want to add a new firm (customer) with its legal details (name, address, tax number, email) so that I can centralise information.
- **US-CORE-02:** As an accountant, I want to associate multiple contacts with the same firm so that I can manage several points of contact.
- **US-CORE-03:** As an accountant, I want to create items with code, designation, pre-tax price and category so that I can easily select them in quotes/invoices.
- **US-CORE-04:** As an administrator, I want to define VAT rates (e.g., 19%, 7%, 0%) with their labels so that I can apply correct taxation.

2.3.3 Theme: Quotes and Invoicing

- **US-INV-01:** As a sales representative, I want to create a quote by selecting a customer, adding items with quantities and possible discounts, so that I can formalise an offer.
- **US-INV-02:** As an accountant, I want to convert an accepted quote into an invoice with one click, preserving the same lines and sequential numbering, so that I can speed up the process.
- **US-INV-03:** As an accountant, I want to view the list of invoices with their statuses (paid, partially paid, unpaid) so that I can track receivables.
- **US-INV-04:** As an accountant, I want to record a payment (amount, date, method) for an invoice, and see the remaining balance update automatically.
- **US-INV-05:** As an accountant, I want to generate a PDF file of an invoice or quote, with a professional layout including the logo and legal notices.

2.3.4 Theme: AI Assistant

- **US-AI-01:** As a user, I want to converse with an integrated chatbot to ask questions about invoice statuses, customers, or to obtain advice.
- **US-AI-02:** As an accountant, I want the assistant to automatically analyse an invoice and alert me to anomalies (suspicious amount, duplicate) or opportunities (unapplied discount).
- **US-AI-03:** As a financial manager, I want the assistant to predict the payment delay risk for each issued invoice, so that I can prioritise reminders.
- **US-AI-04:** As a user, I want an intelligent natural language search (e.g., "unpaid invoices older than 30 days for customer X") that interprets my query and displays the results.
- **US-AI-05:** As a sales representative, I want to receive automatic suggestions for complementary items when creating a quote, based on the customer's history.

2.3.5 Theme: Security and Audit

- **US-SEC-01:** As an administrator, I want all create, modify, delete actions on invoices to be recorded in timestamped logs with the user identifier.
- **US-SEC-02:** As an administrator, I want to consult the log console filtered by date, user and action type.

2.3.6 Theme: Reports and Export

- **US-REP-01:** As an accountant, I want to export a monthly turnover report, detailed by customer, in Excel or CSV format.
- **US-REP-02:** As an administrator, I want a dashboard displaying key indicators (total turnover, number of invoices, quote-to-invoice conversion rate).

2.4 Sprint Planning and Backlog

The product backlog was established by prioritising features according to business value and technical dependencies. Stories were estimated in complexity points (1, 2, 3, 5, 8) using the planning poker method.

2.4.1 Sprint 1: Foundation and Security (3 weeks)

Goal: Have an authenticated platform with user and role management.

- US-AUTH-01 (5 pts): JWT login.
- US-AUTH-02 (5 pts): User management (CRUD).
- US-AUTH-03 (2 pts): Account deactivation.
- US-AUTH-04 (2 pts): Password change.
- US-SEC-01 (3 pts): Basic system logs (authentication).
- Infrastructure: Set up NestJS + Next.js project, MySQL connection, initial models.
- **Deliverable:** Minimal application with login page, simple dashboard, and admin interface to manage users.

2.4.2 Sprint 2: Core Data (3 weeks)

Goal: Enable complete management of reference data (customers, items, taxes).

- US-CORE-01 (3 pts): CRUD for Firms.
- US-CORE-02 (2 pts): Contact management.
- US-CORE-03 (5 pts): CRUD for Items (with categories).
- US-CORE-04 (2 pts): Tax configuration.

- US-REP-02 (3 pts): Basic dashboard with KPIs (number of customers, items).
- **Deliverable:** Functional modules for core data management, accessible according to roles.

2.4.3 Sprint 3: Invoicing Cycle, AI and PDF (3 weeks)

Goal: Integrate the core invoicing business logic, the AI assistant, and advanced features.

- US-INV-01 (5 pts): Quote creation.
- US-INV-02 (3 pts): Quote → invoice conversion.
- US-INV-03 (2 pts): Invoice list with statuses.
- US-INV-04 (3 pts): Payment recording and balance update.
- US-INV-05 (5 pts): PDF generation (customisable template).
- US-AI-01 (8 pts): Chatbot integration (local Ollama + RAG on invoice data).
- US-AI-02 (5 pts): Anomaly detection (rules + lightweight ML, e.g., Isolation Forest).
- US-AI-03 (5 pts): Payment delay prediction (logistic regression model).
- US-AI-04 (3 pts): Natural language search.
- US-AI-05 (3 pts): Complementary item suggestions (collaborative filtering or rule-based).
- US-REP-01 (2 pts): Monthly turnover export.
- **Deliverable:** Complete platform, tested, with functional AI assistant, API documentation and installation guide.

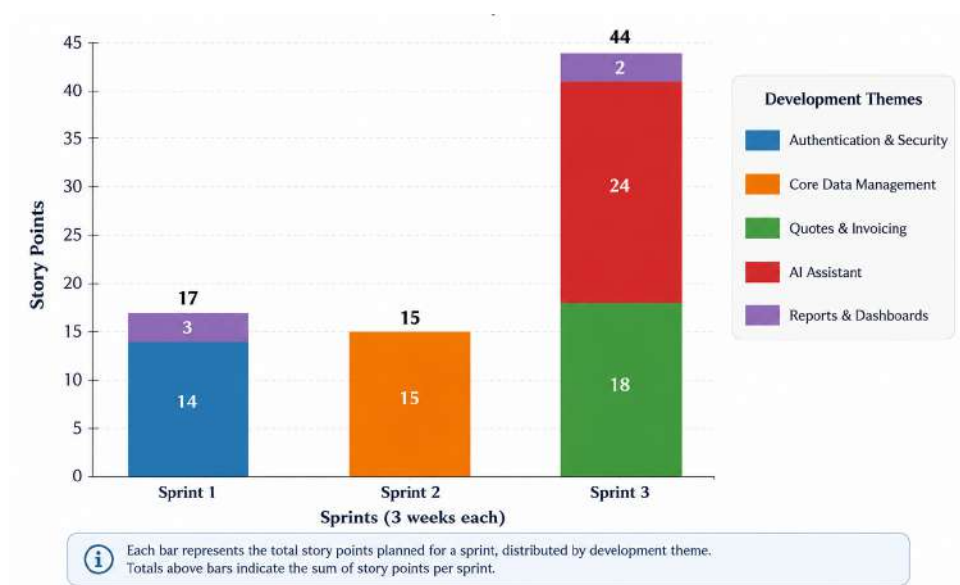


Figure 2.1: User story distribution per sprint

2.5 Chapter Conclusion

This chapter formalised user expectations through precise user stories and structured the work into three iterative sprints. The access rights matrix clarifies the scope of each role, ensuring separation of concerns and appropriate security. With a clear backlog, we can now move on to the detailed system design phase.

Chapter 3

System Architecture and Design

3.1 Introduction

This chapter presents the detailed design of the platform, based on UML (Unified Modeling Language) diagrams and software architecture principles. We describe the static view (class diagram), dynamic views (sequence diagrams, activity diagram), and the deployment architecture. This modelling step is crucial to ensure consistency between requirements and implementation.

3.2 Use Case Diagrams

Use cases illustrate the interactions between actors and the system. We identified about twenty main use cases; the most representative ones are presented below.

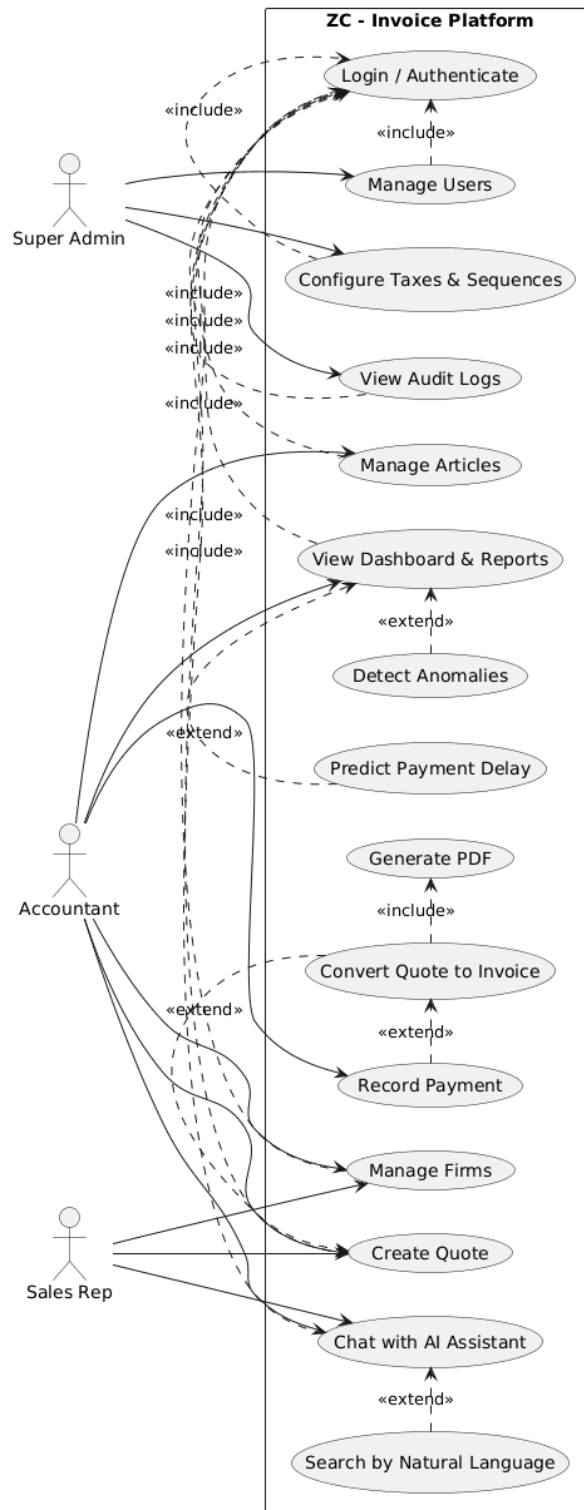


Figure 3.1: Overall use case diagram

3.2.1 Textual Description of a Use Case: "Convert Quote to Invoice"

- **Primary actor:** Accountant.
- **Preconditions:** The quote exists and has the status "Accepted".
- **Trigger:** The user selects a quote from the list and clicks "Convert".

- **Nominal scenario:**

1. The system displays a summary of the quote.
2. The user confirms the conversion.
3. The system creates an invoice by copying the line items, generates a new sequential number (according to the configured sequence), and assigns the status "Unpaid".
4. The system marks the quote as "Converted".
5. The system displays a success message and redirects to the newly created invoice.

- **Postconditions:** The invoice is persisted, and the sequences are incremented.

3.3 Class Diagram

The UML class diagram represents the business entities and their relationships. The main classes are: User, Role, Firm, Contact, Article, Tax, Quote, Invoice, Payment, Log, AIRequest, AISuggestion.

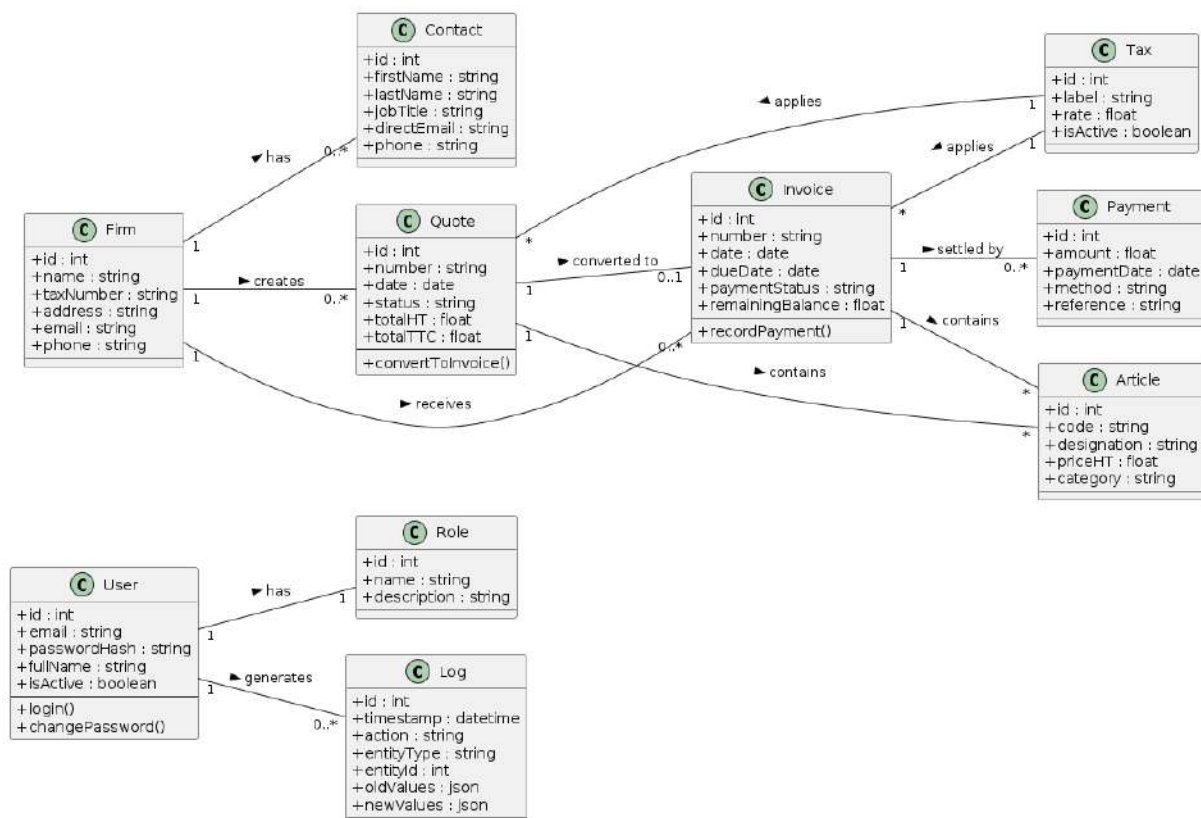


Figure 3.2: Simplified class diagram

Table 3.1: Description of the main classes

Class	Description
User	System user, linked to a Role. Contains email, hashed password, name, active flag.
Role	Defines permissions (via a Permission table in Many-to-Many relationship).
Firm	Customer (company): company name, address, tax number, email, phone.
Contact	Contact person within a firm: title, first name, last name, job title, direct email.
Article	Product or service sold: code, designation, pre-tax price, category.
Tax	VAT rate: label, rate (e.g., 0.19), isActive.
Quote	Quote: number, date, status (draft, sent, accepted, rejected, converted), total pre-tax, total tax-inclusive, relationship with Firm and line items (QuoteLine).
Invoice	Invoice: similar to Quote but with payment statuses, due date, remaining balance.
Payment	Payment: amount, date, method (cash, transfer, cheque), reference.
Log	Audit log: user, action, affected entity, timestamp, old/new values (JSON).

3.4 Sequence Diagrams

Sequence diagrams illustrate the temporal interactions between objects to carry out a scenario. We detail two of them: authentication and invoice creation.

3.4.1 JWT Authentication

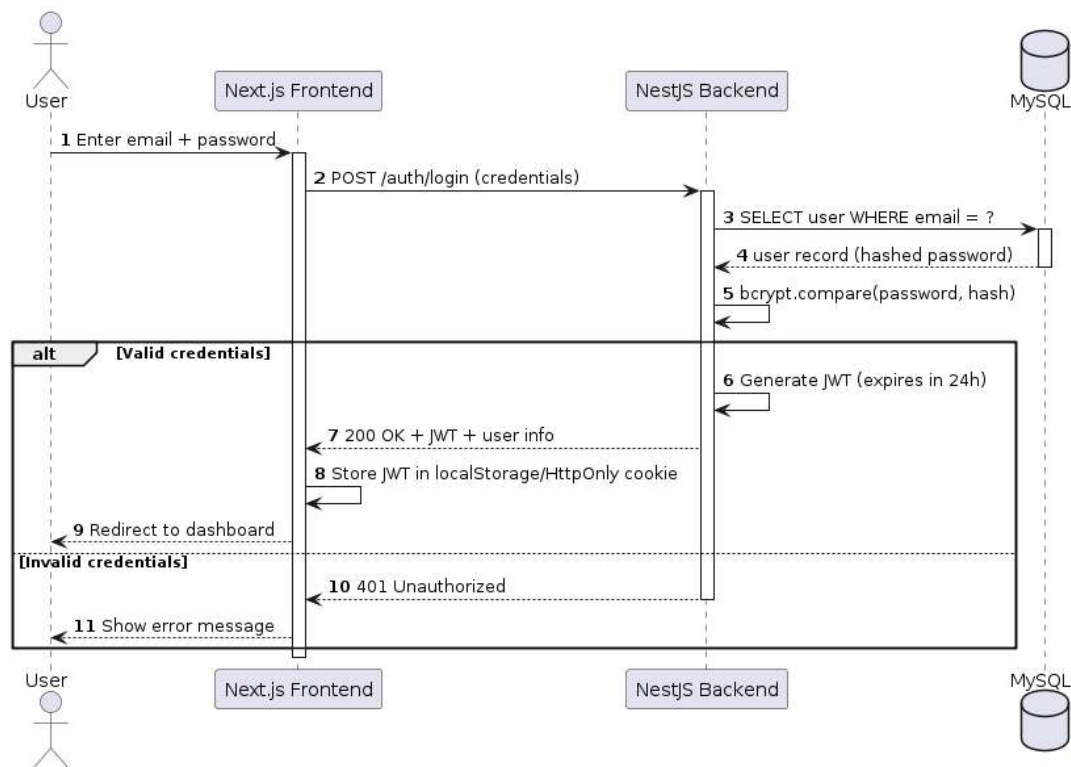


Figure 3.3: Sequence diagram of the authentication process

3.4.2 Creating an Invoice from a Quote

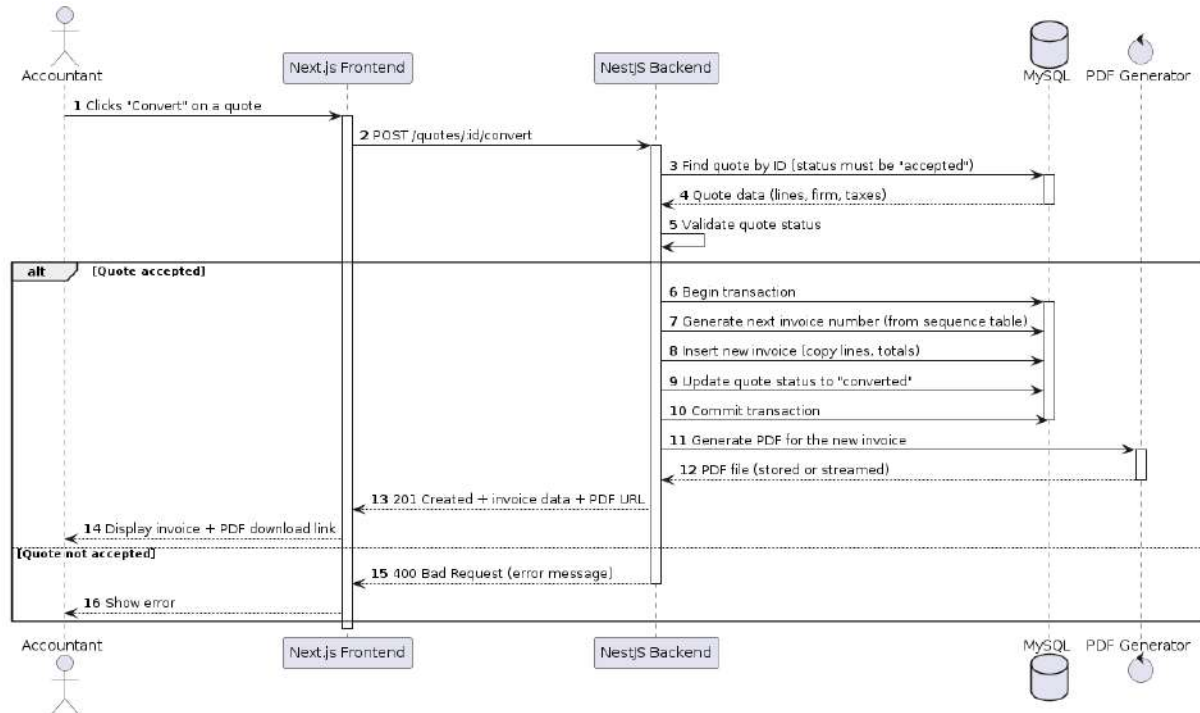


Figure 3.4: Sequence diagram of quote → invoice conversion

3.5 Activity Diagram

The activity diagram models the control flow of the document lifecycle (quote or invoice).

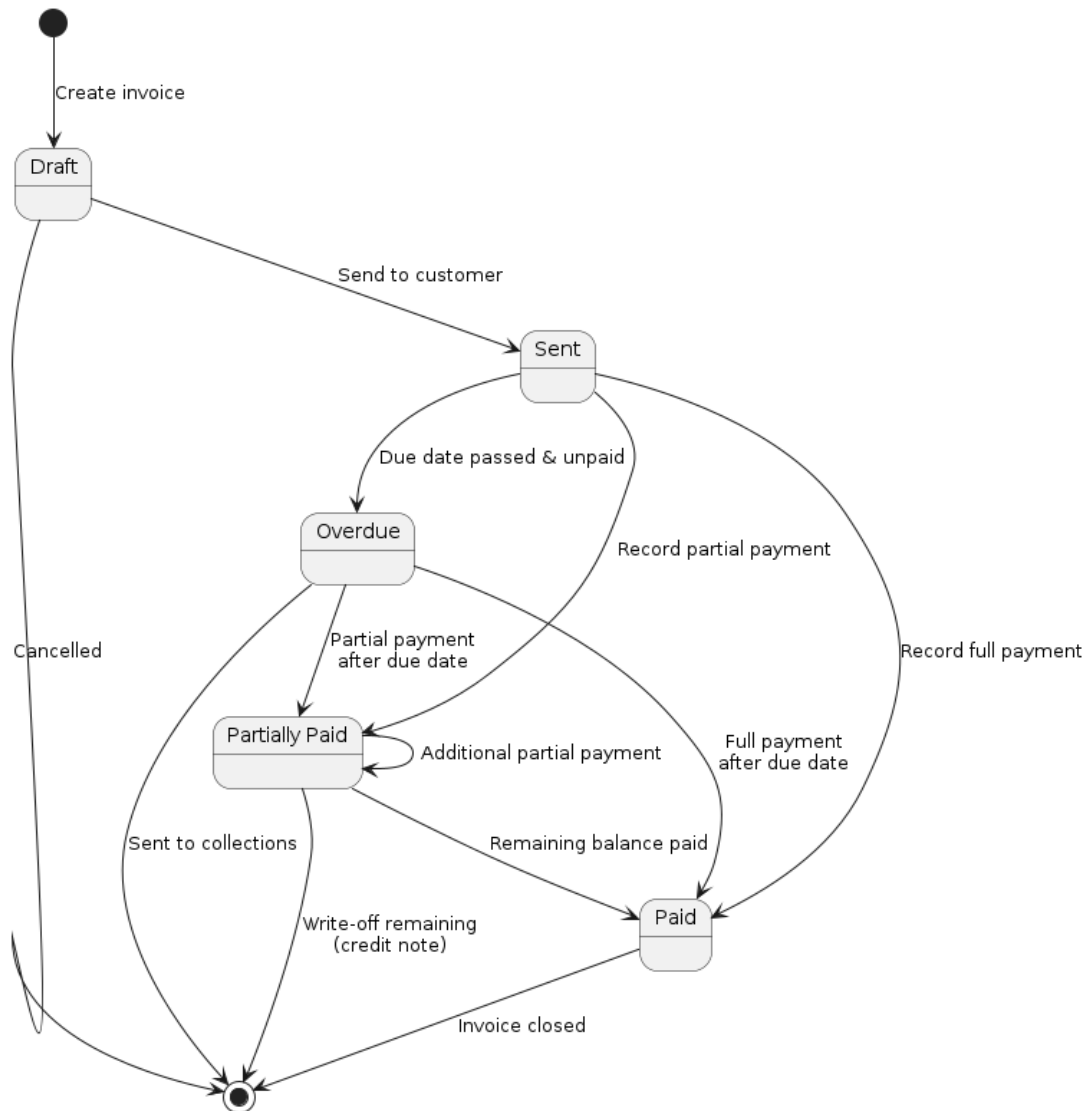


Figure 3.5: Activity diagram of the document lifecycle

3.6 Component Diagram

The software architecture is divided into independent components (NestJS modules). The component diagram shows the dependencies.

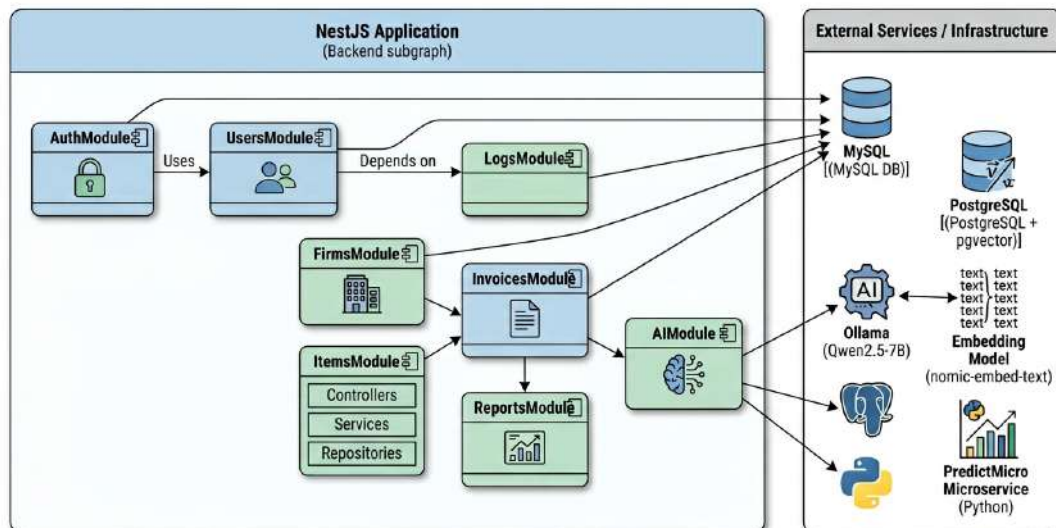


Figure 3.6: Component diagram (backend)

3.7 Deployment Diagram

The system is deployed on a virtualised infrastructure (Docker) with the following nodes: client (browser), web server (Nginx), application server (NestJS), database server (MySQL), and a dedicated server for the local AI (Ollama with Qwen2.5-7B, embedding model, and pgvector). The figure below illustrates this deployment.

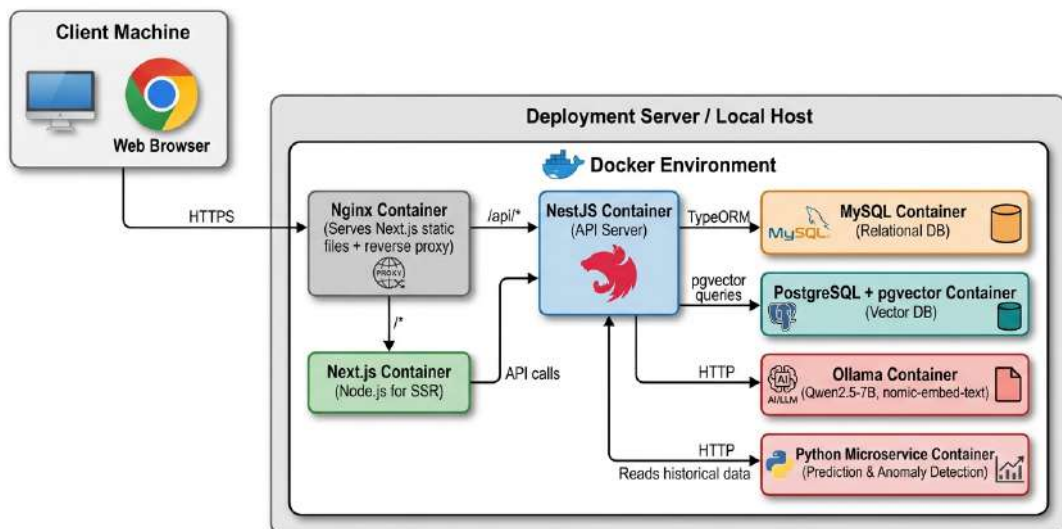


Figure 3.7: Deployment diagram

3.8 Database Modelling (ERD)

The entity-relationship diagram (ERD) forms the basis of the relational schema. The main entities and their relationships are represented.

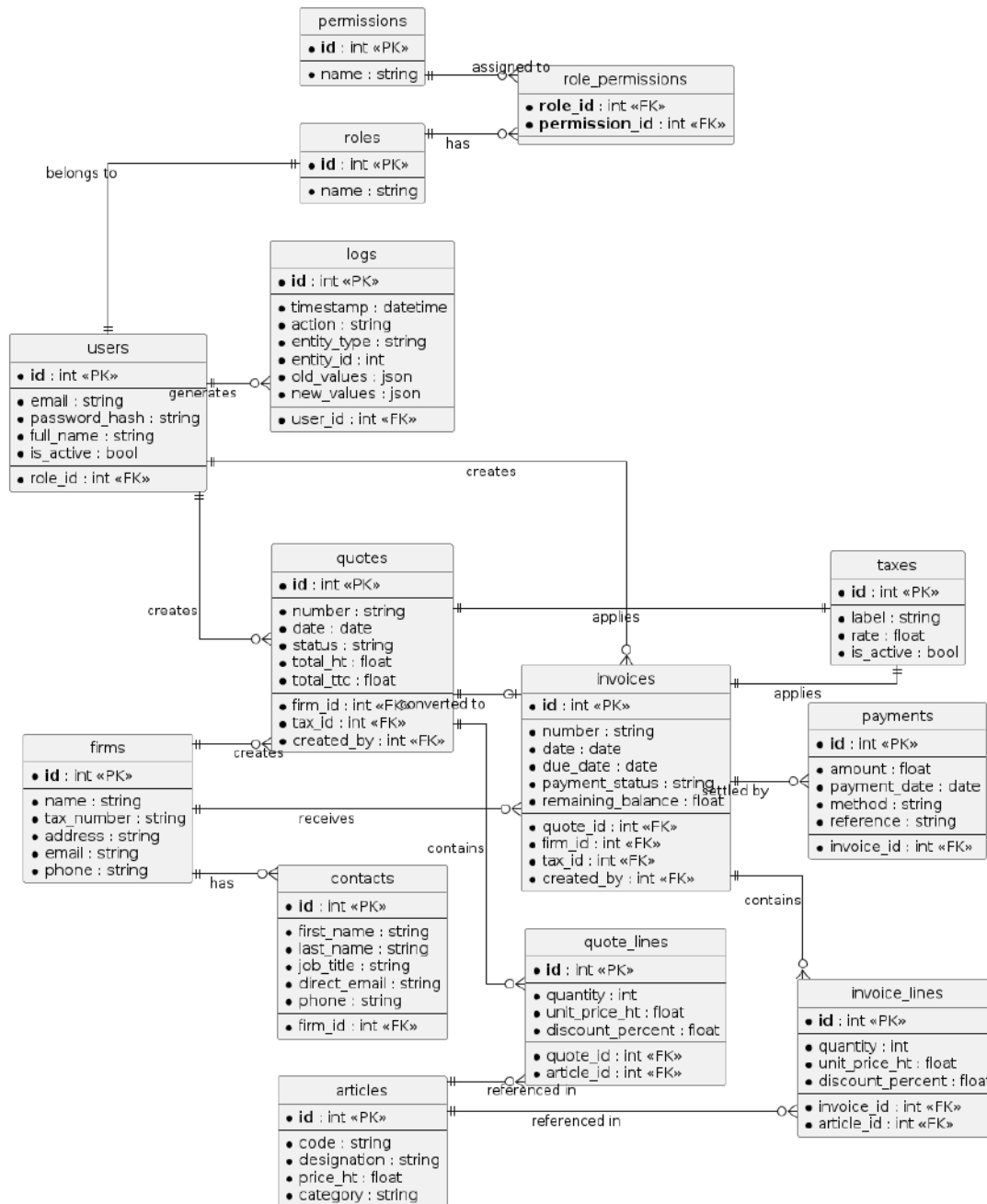


Figure 3.8: Entity-Relationship Diagram (ERD)

3.9 Compliance with Architecture Principles

The design respects the following principles:

- **SOLID**: classes are open for extension but closed for modification (OCP principle). Dependencies are injected (DIP).
- **Separation of concerns**: the controller layer contains no business logic; services are reusable.
- **Clean Architecture**: business entities are independent of external frameworks. Use cases (services) orchestrate the repositories.

3.10 Chapter Conclusion

The UML diagrams produced provide a comprehensive view of the platform's design. They will serve as a reference for developers and facilitate future maintenance. The next chapter dives into the implementation of the NestJS backend.

Chapter 4

Backend Development with NestJS

4.1 Introduction

This chapter details the technical implementation of the backend. We describe the modular structure, controllers, services, TypeORM integration, DTO management, validation, and the setup of guards. Significant code excerpts illustrate the key concepts.

4.2 Project Structure

The NestJS project follows the module convention. The main directory tree is as follows:

```
src/
  main.ts
  app.module.ts
  auth/                                # Authentication
    auth.controller.ts
    auth.service.ts
    strategies/ (jwt.strategy.ts, local.strategy.ts)
    guards/ (jwt-auth.guard.ts, roles.guard.ts)
  users/                               # User management
    users.module.ts
    users.controller.ts
    users.service.ts
    entities/user.entity.ts
  roles/                               # Roles and permissions
  firms/                               # Customers
  contacts/                            # Contacts
  articles/                            # Item catalogue
  taxes/                               # Taxes
  quotes/                              # Quotes
  invoices/                            # Invoices
  payments/                            # Payments
  logs/                                # Audit logs
  ai/                                  # AI assistant
    ai.controller.ts
    ai.service.ts
    rag/ (embeddings, vector store)
    prompts/ (templates)
  dto/
```

`common/``# Global DTOs, interceptors, pipes`

4.3 Setting up Authentication (JWT)

The authentication module uses Passport strategies integrated into NestJS.

4.3.1 JWT Strategy

Listing 4.1: `jwt.strategy.ts`

```
1  import { ExtractJwt, Strategy } from 'passport-jwt';
2  import { PassportStrategy } from '@nestjs/passport';
3  import { Injectable } from '@nestjs/common';
4  import { ConfigService } from '@nestjs/config';
5
6  @Injectable()
7  export class JwtStrategy extends PassportStrategy(Strategy) {
8    constructor(private configService: ConfigService) {
9      super({
10        jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
11        ignoreExpiration: false,
12        secretOrKey: configService.get('JWT_SECRET'),
13      });
14    }
15
16    async validate(payload: any) {
17      // Returns the user attached to the request
18      return { userId: payload.sub, email: payload.email, role:
19        payload.role };
20    }
21  }
```

4.3.2 Authentication Service

The service handles credential verification and token generation.

Listing 4.2: `auth.service.ts` (excerpt)

```
1  import * as bcrypt from 'bcrypt';
2  import { JwtService } from '@nestjs/jwt';
3
4  @Injectable()
5  export class AuthService {
6    constructor(
7      private usersService: UsersService,
8      private jwtService: JwtService,
9    ) {}
10
11    async validateUser(email: string, pass: string): Promise<any>
12    {
```

```

12     const user = await this.usersService.findByEmail(email);
13     if (user && await bcrypt.compare(pass, user.passwordHash))
14     {
15         const { passwordHash, ...result } = user;
16         return result;
17     }
18     return null;
19 }
20
21 async login(user: any) {
22     const payload = { email: user.email, sub: user.id, role:
23         user.role.name };
24     return {
25         access_token: this.jwtService.sign(payload),
26         user: { id: user.id, email: user.email, role: user.role.
27             name }
28     };
29 }
30 }

```

4.3.3 Authentication Controller

Listing 4.3: auth.controller.ts

```

1 import { Controller, Post, Body, UseGuards, Request } from '@nestjs/common';
2 import { AuthGuard } from '@nestjs/passport';
3
4 @Controller('auth')
5 export class AuthController {
6     constructor(private authService: AuthService) {}
7
8     @UseGuards(AuthGuard('local'))
9     @Post('login')
10    async login(@Request() req) {
11        return this.authService.login(req.user);
12    }
13 }

```

4.4 Role and Permission Management (RBAC)

We implemented a custom RolesGuard that reads the role from the JWT token and compares it with the allowed roles via the `@Roles()` decorator.

4.4.1 Roles Decorator

Listing 4.4: roles.decorator.ts

```

1  import { SetMetadata } from '@nestjs/common';
2  export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

```

4.4.2 RolesGuard

Listing 4.5: roles.guard.ts

```

1  import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
2  import { Reflector } from '@nestjs/core';
3
4  @Injectable()
5  export class RolesGuard implements CanActivate {
6      constructor(private reflector: Reflector) {}
7
8      canActivate(context: ExecutionContext): boolean {
9          const requiredRoles = this.reflector.get<string[]>('roles', context.getHandler());
10         if (!requiredRoles) return true;
11         const { user } = context.switchToHttp().getRequest();
12         return requiredRoles.some((role) => user.role === role);
13     }
14 }

```

4.4.3 Usage in a Controller

Listing 4.6: users.controller.ts (excerpt)

```

1  @Controller('users')
2  @UseGuards(JwtAuthGuard, RolesGuard)
3  export class UsersController {
4      @Get()
5      @Roles('admin')
6      findAll() {
7          return this.userService.findAll();
8      }
9  }

```

4.5 Invoicing Module

The InvoicesModule is the core business module. Below is an excerpt from the invoice creation service.

Listing 4.7: invoices.service.ts (excerpt)

```

1  @Injectable()
2  export class InvoicesService {
3      constructor(

```

```

4      @InjectRepository(Invoice)
5      private invoiceRepository: Repository<Invoice>,
6      @InjectRepository(Quote)
7      private quoteRepository: Repository<Quote>,
8      private sequenceService: SequenceService,
9      ) {}

10
11      async createFromQuote(quoteId: number): Promise<Invoice> {
12          const quote = await this.quoteRepository.findOne({ where: {
13              id: quoteId }, relations: ['lines', 'firm'] });
14          if (!quote) throw new NotFoundException('Quote not found');
15          if (quote.status !== 'accepted') throw new
16              BadRequestException('Quote must be accepted');
17
18          // Generate invoice number via sequence
19          const invoiceNumber = await this.sequenceService.
20              generateNextNumber('invoice');
21
22          const invoice = this.invoiceRepository.create({
23              number: invoiceNumber,
24              date: new Date(),
25              dueDate: new Date(new Date().setDate(new Date().getDate()
26                  + 30)),
27              firm: quote.firm,
28              totalHT: quote.totalHT,
29              totalTTC: quote.totalTTC,
30              status: 'unpaid',
31              remainingAmount: quote.totalTTC,
32              lines: quote.lines.map(line => ({ ...line, id: undefined
33                  }))) // copy lines
34          });
35
36          const savedInvoice = await this.invoiceRepository.save(
37              invoice);
38          // Mark quote as converted
39          quote.status = 'converted';
40          await this.quoteRepository.save(quote);
41
42          return savedInvoice;
43      }
44  }

```

4.6 Data Validation (DTOs and Pipes)

We use the `class-validator` library with DTOs to validate incoming requests.

Listing 4.8: create-invoice.dto.ts

```

1      import { IsNotEmpty, IsPositive, IsDateString, IsArray,
2          ValidateNested } from 'class-validator';
3      import { Type } from 'class-transformer';

```

```

3
4 export class CreateInvoiceDto {
5   @IsNotEmpty()
6   firmId: number;
7
8   @IsDateString()
9   dueDate: string;
10
11   @IsArray()
12   @ValidateNested({ each: true })
13   @Type(() => InvoiceLineDto)
14   lines: InvoiceLineDto[];
15 }
16
17 export class InvoiceLineDto {
18   @IsNotEmpty()
19   articleId: number;
20
21   @IsPositive()
22   quantity: number;
23
24   @IsPositive()
25   unitPriceHT: number;
26
27   taxId: number;
28 }

```

In the controller, we use `@Body()` `createInvoiceDto`: `CreateInvoiceDto` and a global validation pipe.

4.7 Error Handling and Logging

A global exception filter catches errors and records them in the system logs.

Listing 4.9: `http-exception.filter.ts`

```

1 import { ExceptionFilter, Catch, ArgumentsHost, HttpException,
2   Logger } from '@nestjs/common';
3
4 @Catch(HttpException)
5 export class HttpExceptionFilter implements ExceptionFilter {
6   private readonly logger = new Logger(HttpExceptionFilter.name);
7
8   catch(exception: HttpException, host: ArgumentsHost) {
9     const ctx = host.switchToHttp();
10    const response = ctx.getResponse();
11    const request = ctx.getRequest();
12    const status = exception.getStatus();
13    const message = exception.message;

```

```
14     this.logger.error(`${request.method} ${request.url} - ${  
15         status} - ${message}`);  
16  
17     response.status(status).json({  
18         statusCode: status,  
19         timestamp: new Date().toISOString(),  
20         path: request.url,  
21         message,  
22     });  
23 }
```

4.8 API Documentation with Swagger

NestJS integrates easily with Swagger via the `@nestjs/swagger` package. We annotated controllers and DTOs to automatically generate interactive documentation.

Listing 4.10: main.ts (swagger)

```
1  import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger'  
2  
3  const config = new DocumentBuilder()  
4  .setTitle('ZC - invoice API')  
5  .setDescription('Invoicing platform API')  
6  .setVersion('1.0')  
7  .addBearerAuth()  
8  .build();  
9  const document = SwaggerModule.createDocument(app, config);  
10 SwaggerModule.setup('api/docs', app, document);
```

4.9 Chapter Conclusion

The backend implementation follows NestJS best practices: modularity, dependency injection, input validation, and automatic documentation. The core features (authentication, RBAC, invoice management) are operational. The next chapter focuses on frontend development with Next.js.

Chapter 5

Frontend Development with Next.js

5.1 Introduction

The user interface is a key element for platform adoption. This chapter describes the Next.js project structure, state management, integration with the backend API, dynamic form creation, navigation (App Router), and reusable components. Next.js was chosen for its hybrid rendering capabilities (SSR, SSG, CSR), built-in routing, and optimised performance.

5.2 Next.js Project Structure

The project uses the ****App Router**** (Next.js 14+). The architecture is feature-based:

```
1 src /
2     app /
3         layout.tsx           # Root layout (
AuthProvider, UI shell)
4         page.tsx             # Redirect to /dashboard
5         login /
6             page.tsx         # Login page
7         dashboard /
8             page.tsx         # Dashboard page
9         invoices /
10            page.tsx          # List invoices
11            [id] /
12                page.tsx      # Invoice detail page
13        quotes /
14            page.tsx          # List quotes
15            create /
16                page.tsx      # Create quote page
17        ai /
18            page.tsx          # AI chatbot page
19        api /                 # Optional API routes (
proxy, etc.)
20        components /         # Reusable components (
Button, Modal, Table)
21        contexts /           # React contexts (
AuthContext, NotificationContext)
22        hooks /              # Custom hooks (useAuth,
useInvoices)
23        lib /                # API client, utilities
```

```

24         apiClient.ts           # Axios instance with
interceptors
25         utils.ts
26         types/                 # TypeScript definitions
27         styles/                # Global styles (Tailwind)

```

5.3 Frontend Authentication Management

We use a **client component** for the AuthContext (marked with “use client”). The context stores the JWT token and user information, and persists the token in ‘localStorage’.

Listing 5.1: contexts/AuthContext.tsx

```

1  'use client';
2
3  import React, { createContext, useState, useContext, useEffect
   } from 'react';
4  import { useRouter } from 'next/navigation';
5  import { login as apiLogin } from '@lib/api/authAPI';
6
7  interface AuthContextType {
8      user: User | null;
9      token: string | null;
10     login: (email: string, password: string) => Promise<void>;
11     logout: () => void;
12     hasRole: (role: string) => boolean;
13 }
14
15 const AuthContext = createContext<AuthContextType | undefined>(
   undefined);
16
17 export const AuthProvider = ({ children }: { children: React.
   ReactNode }) => {
18     const [user, setUser] = useState<User | null>(null);
19     const [token, setToken] = useState<string | null>(null);
20     const router = useRouter();
21
22     useEffect(() => {
23         // Load token from localStorage on mount
24         const storedToken = localStorage.getItem('token');
25         if (storedToken) {
26             setToken(storedToken);
27             const userData = decodeToken(storedToken);
28             setUser(userData);
29         }
30     }, []);
31
32     const login = async (email: string, password: string) => {
33         const response = await apiLogin(email, password);
34         const { access_token, user: userData } = response;

```

```
35     setToken(access_token);
36     localStorage.setItem('token', access_token);
37     setUser(userData);
38     router.push('/dashboard');
39   };
40
41   const logout = () => {
42     setToken(null);
43     localStorage.removeItem('token');
44     setUser(null);
45     router.push('/login');
46   };
47
48   const hasRole = (role: string) => user?.role === role;
49
50   return (
51     <AuthContext.Provider value={{ user, token, login, logout,
52       hasRole }}>
53       {children}
54     </AuthContext.Provider>
55   );
56
57   export const useAuth = () => {
58     const context = useContext(AuthContext);
59     if (!context) throw new Error('useAuth must be used within
60       AuthProvider');
61     return context;
62   };
```

5.4 API Integration with Axios

We create a centralised Axios client (used in client components) that automatically attaches the JWT token.

Listing 5.2: lib/apiClient.ts

```
1   import axios from 'axios';
2
3   const apiClient = axios.create({
4     baseURL: process.env.NEXT_PUBLIC_API_URL,
5     headers: { 'Content-Type': 'application/json' },
6   });
7
8   apiClient.interceptors.request.use((config) => {
9     if (typeof window !== 'undefined') {
10       const token = localStorage.getItem('token');
11       if (token) {
12         config.headers.Authorization = `Bearer ${token}`;
13       }
14     }
15   });
```

```
15     return config;
16   });
17
18   export default apiClient;
```

5.4.1 Invoice Service (Client-side)

Listing 5.3: lib/api/invoiceAPI.ts

```
1  import apiClient from '../apiClient';
2  import { Invoice } from '@types/Invoice';
3
4  export const getInvoices = async (): Promise<Invoice[]> => {
5    const response = await apiClient.get('/invoices');
6    return response.data;
7  };
8
9  export const getInvoiceById = async (id: number): Promise<
10    Invoice> => {
11    const response = await apiClient.get(`/invoices/${id}`);
12    return response.data;
13  };
14
15  export const createInvoiceFromQuote = async (quoteId: number):
16    Promise<Invoice> => {
17    const response = await apiClient.post(`/invoices/convert/${quoteId}`);
18    return response.data;
19  };
```

5.5 Key Pages (App Router)

5.5.1 Login Page (Client Component)

Listing 5.4: app/login/page.tsx

```
1  'use client';
2
3  import { useState } from 'react';
4  import { useAuth } from '@contexts/AuthContext';
5
6  export default function LoginPage() {
7    const [email, setEmail] = useState('');
8    const [password, setPassword] = useState('');
9    const [error, setError] = useState('');
10    const { login } = useAuth();
11
12    const handleSubmit = async (e: React.FormEvent) => {
13      e.preventDefault();
```

```

14     try {
15       await login(email, password);
16     } catch (err) {
17       setError('Invalid email or password');
18     }
19   };
20
21   return (
22     <div className="min-h-screen flex items-center justify-center
23       bg-gray-50">
24     <form onSubmit={handleSubmit} className="bg-white p-8 rounded
25       shadow-md w-96">
26     <h2 className="text-2xl font-bold mb-6">Login</h2>
27     {error && <div className="text-red-500 mb-4">{error}</div>}
28     <div className="mb-4">
29     <label className="block text-gray-700">Email</label>
30     <input type="email" value={email} onChange={(e) => setEmail(e
31       .target.value)} className="w-full border rounded px-3 py-2
32       " required />
33     </div>
34     <div className="mb-6">
35     <label className="block text-gray-700">Password</label>
36     <input type="password" value={password} onChange={(e) =>
37       setPassword(e.target.value)} className="w-full border
38       rounded px-3 py-2" required />
39     </div>
40     <button type="submit" className="w-full bg-blue-600 text-
41       white py-2 rounded hover:bg-blue-700">Sign in</button>
42     </form>
43     </div>
44   );
45 }

```

5.5.2 Dashboard Page (Client Component with Data Fetching)

The dashboard fetches data from the backend API.

Listing 5.5: app/dashboard/page.tsx

```

1   'use client';
2
3   import { useEffect, useState } from 'react';
4   import { getDashboardStats } from '@lib/api/dashboardAPI';
5   import { useAuth } from '@contexts/AuthContext';
6
7   export default function DashboardPage() {
8     const [stats, setStats] = useState({ totalRevenue: 0,
9       unpaidInvoices: 0, totalClients: 0 });
10    const { user } = useAuth();
11
12    useEffect(() => {
13      const fetchStats = async () => {

```

```

13         const data = await getDashboardStats();
14         setStats(data);
15     };
16     fetchStats();
17 }, []);
18
19 return (
20 <div className="p-6">
21 <h1 className="text-3xl font-bold mb-6">Dashboard</h1>
22 <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
23 <div className="bg-white p-4 rounded shadow">
24 <h3 className="text-gray-500">Monthly Turnover</h3>
25 <p className="text-2xl font-bold">{stats.totalRevenue} TND</p>
26 </div>
27 <div className="bg-white p-4 rounded shadow">
28 <h3 className="text-gray-500">Unpaid Invoices</h3>
29 <p className="text-2xl font-bold text-red-600">{stats.
    unpaidInvoices}</p>
30 </div>
31 <div className="bg-white p-4 rounded shadow">
32 <h3 className="text-gray-500">Active Clients</h3>
33 <p className="text-2xl font-bold">{stats.totalClients}</p>
34 </div>
35 </div>
36 </div>
37 );
38 }

```

5.5.3 Quote Creation Page (React Hook Form + useFieldArray)

Because of dynamic form fields, this is a client component.

Listing 5.6: app/quotes/create/page.tsx (excerpt)

```

1  'use client';
2
3  import { useForm, useFieldArray } from 'react-hook-form';
4  import { createQuote } from '@/lib/api/quoteAPI';
5
6  type QuoteFormData = {
7      firmId: number;
8      lines: { articleId: number; quantity: number; unitPriceHT:
          number; taxId: number }[];
9  };
10
11  export default function QuoteCreatePage() {
12      const { register, control, handleSubmit } = useForm<
          QuoteFormData>({
13          defaultValues: { lines: [{ articleId: 0, quantity: 1,
              unitPriceHT: 0, taxId: 1 }] }
14      });

```

```

15     const { fields, append, remove } = useFieldArray({ control,
16         name: 'lines' });
17
18     const onSubmit = async (data: QuoteFormData) => {
19         await createQuote(data);
20         // redirect to quotes list
21     };
22
23     return (
24         <form onSubmit={handleSubmit(onSubmit)} className="p-6 max-w
25             -4xl mx-auto">
26             <h1 className="text-2xl font-bold mb-4">Create Quote</h1>
27             <select {...register('firmId', { required: true })} className=
28                 "border p-2 mb-4 w-full">
29                 <option value="">Select a client</option>
30                 {/* firms options fetched from API */}
31             </select>
32             {fields.map((field, idx) => (
33                 <div key={field.id} className="flex gap-2 mb-2">
34                 <select {...register('lines.${idx}.articleId')} className="
35                     border p-2">...</select>
36                 <input type="number" {...register('lines.${idx}.quantity')}
37                     className="border p-2 w-24" />
38                 <input type="number" {...register('lines.${idx}.unitPriceHT
39                     ')} className="border p-2 w-32" />
40                 <button type="button" onClick={() => remove(idx)} className=
41                     "bg-red-500 text-white px-2">Remove</button>
42                 </div>
43             ))}
44             <button type="button" onClick={() => append({ articleId: 0,
45                 quantity: 1, unitPriceHT: 0, taxId: 1 })} className="bg-
46                 gray-200 px-4 py-2 mr-2">Add line</button>
47             <button type="submit" className="bg-blue-600 text-white px-4
48                 py-2">Create quote</button>
49         </form>
50     );
51 }

```

5.6 PDF Generation (Client-side)

The frontend calls the backend endpoint and triggers a download.

Listing 5.7: /page.tsx (excerpt)]app/invoices/[id]/page.tsx (excerpt)

```

1     'use client';
2
3     import apiClient from '@/lib/apiClient';
4
5     const handleDownloadPDF = async (invoiceId: number) => {
6         const response = await apiClient.get(`/invoices/${invoiceId}/
7             pdf`, { responseType: 'blob' });
8     };

```

```

7      const url = window.URL.createObjectURL(new Blob([response.
      data], { type: 'application/pdf' }));
8      const link = document.createElement('a');
9      link.href = url;
10     link.setAttribute('download', `invoice_${invoiceId}.pdf`);
11     document.body.appendChild(link);
12     link.click();
13     link.remove();
14     window.URL.revokeObjectURL(url);
15 };

```

5.7 Server Components for Performance

For pages that do not require client interactivity (e.g., viewing a static invoice list), we can use a **server component** to fetch data directly on the server, improving performance and SEO.

Listing 5.8: app/invoices/page.tsx (server component)

```

1      import { getInvoices } from '@lib/api/invoiceAPI'; // this
      would need to be adapted for server-side
2
3      export default async function InvoicesPage() {
4          // Fetch data on the server (using fetch with full URL)
5          const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/
      invoices`, {
6              headers: { Cookie: cookies().toString() } // if using
      cookie-based token
7          });
8          const invoices = await res.json();
9
10         return (
11             <div className="p-6">
12                 <h1 className="text-2xl font-bold mb-4">Invoices</h1>
13                 <table className="w-full border-collapse border">
14                     <thead>
15                         <tr><th className="border p-2">Number</th><th className="
      border p-2">Client</th><th className="border p-2">Total</
      th></tr>
16                     </thead>
17                     <tbody>
18                         {invoices.map((inv: any) => (
19                             <tr key={inv.id}><td className="border p-2">{inv.number}</
      td><td className="border p-2">{inv.firm.name}</td><td
      className="border p-2">{inv.totalTTC} TND</td></tr>
20                         ))}
21                     </tbody>
22                 </table>
23             </div>
24         );
25     }

```


5.8 Responsive Design with Tailwind CSS

Tailwind CSS is used throughout the project. The root layout includes the ‘AuthProvider’ and a sidebar navigation that collapses on mobile.

Listing 5.9: app/layout.tsx

```
1  import { AuthProvider } from '@contexts/AuthContext';
2  import './globals.css';
3
4  export default function RootLayout({ children }: { children:
5    React.ReactNode }) {
6    return (
7      <html lang="en">
8        <body>
9          <AuthProvider>
10           <div className="flex min-h-screen">
11             <aside className="w-64 bg-gray-800 text-white p-4 hidden md:
12               block">Sidebar</aside>
13             <main className="flex-1 bg-gray-100">{children}</main>
14           </div>
15         </AuthProvider>
16       </body>
17     </html>
18   );
19 }
```

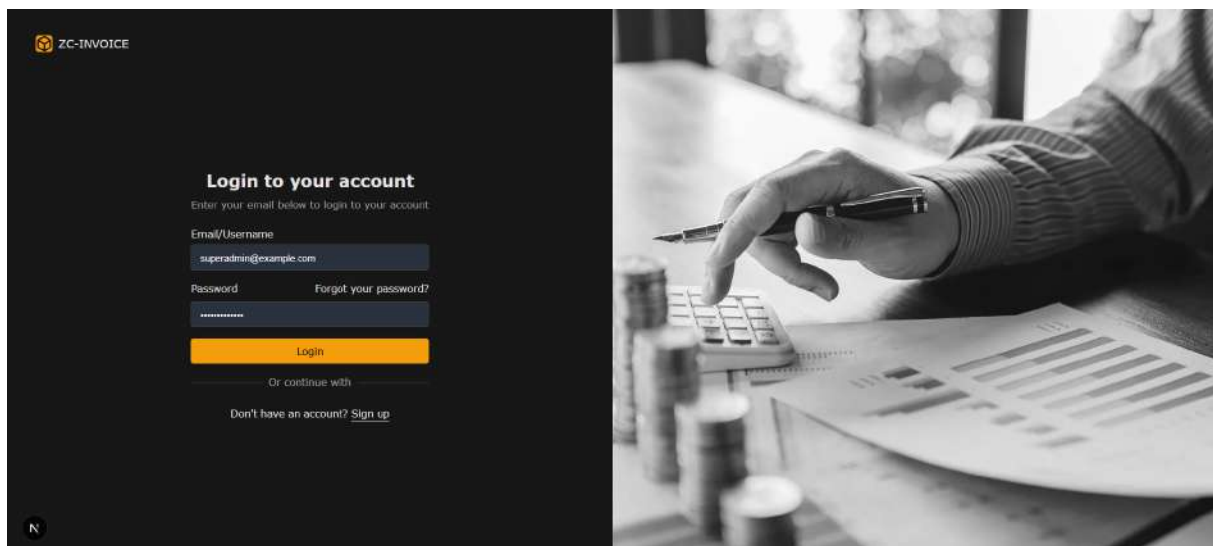


Figure 5.1: Login page

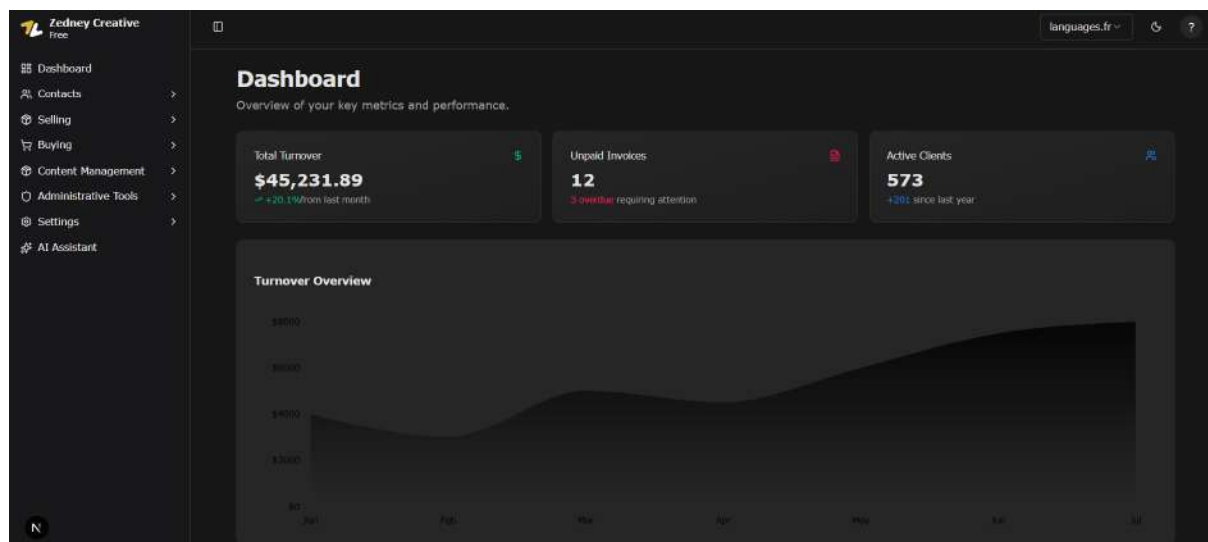


Figure 5.2: dashboard page

The screenshot shows the 'Invoices' management page in the Zedney Creative application. The sidebar is the same as in Figure 5.2, but the 'Selling' menu is expanded. The main header shows 'Selling > Invoices' and 'Manage your invoices'. Below this is a search bar labeled 'datatable.search' and a 'commands.display' button. A table lists five invoices with columns for Number, Date, Firm, Interlocutor, Due Date, Status, Total, Amount Paid, Withholding, and Created At. Each row has a status button (Send, Draft, Validated, Paid, Send) and a three-dot menu icon.

Number	Date	Firm	Interlocutor	Due Date	Status	Total	Amount Paid	Withholding	Created At
INV-2026-001	15-05-2026	Acme Corporation		15-06-2026	Send	1250 \$	\$	\$	—
INV-2026-002	18-05-2026	Globex Inc		18-06-2026	Draft	3401 \$	\$	\$	—
INV-2026-003	20-05-2026	Stark Industries		20-06-2026	Validated	890 \$	\$	\$	—
INV-2026-004	22-05-2026	Wayne Enterprises		22-06-2026	Paid	5500 \$	\$	\$	—
INV-2026-005	25-05-2026	Acme Corporation		25-06-2026	Send	420 \$	\$	\$	—

Figure 5.3: Invoice list with filters

5.9 Chapter Conclusion

The Next.js frontend provides a modern, performant, and responsive user interface. The App Router simplifies routing and layout management, while server components optimise data fetching where appropriate. The integration with the backend API is secured via JWT and Axios interceptors. Key pages (login, dashboard, quote creation, invoice list) are fully functional. The next chapter discusses security and authentication mechanisms in more depth.

Chapter 6

Authentication and Security

6.1 Introduction

Security is a major concern for a platform handling sensitive financial data. This chapter details the mechanisms implemented: JWT authentication, password hashing, role-based access control (RBAC), protection against common attacks (SQL injection, XSS, CSRF), and securing API calls (rate limiting, validation).

6.2 JWT Authentication

As described previously, authentication relies on JSON Web Tokens (JWT). Tokens have a limited lifetime (8 hours) and are signed with a secret key. To prevent token theft, we recommend using in-memory storage (via Next.js context) rather than `localStorage` in production, but for simplicity we stored tokens in `localStorage` with appropriate warnings.

6.2.1 Token Refresh

For enhanced security, a refresh token mechanism could be implemented (not done in the initial version but planned as an improvement).

6.3 Password Hashing (bcrypt)

bcrypt is used to hash passwords with a cost factor of 12, making brute-force attacks very expensive.

Listing 6.1: Excerpt from `users.service.ts`

```
1 import * as bcrypt from 'bcrypt';
2 const saltRounds = 12;
3
4 async create(createUserDto: CreateUserDto) {
5   const hashedPassword = await bcrypt.hash(createUserDto.
6     password, saltRounds);
7   const user = this.userRepository.create({ ...createUserDto,
8     passwordHash: hashedPassword });
9   return this.userRepository.save(user);
10 }
```

6.4 Role-Based Access Control (RBAC)

The backend uses a RolesGuard to restrict access to endpoints. Roles are stored in the database and included in the JWT token. Verification occurs on every protected request.

6.4.1 Example of Admin-Only Route

```
1 @Delete('/:id')
2 @Roles('admin')
3 remove(@Param('id') id: string) {
4     return this.userService.remove(+id);
5 }
```

6.5 Protection Against Attacks

6.5.1 SQL Injection

The use of TypeORM (or any ORM) automatically parameterises queries, preventing SQL injection. For raw queries, we use named parameters.

6.5.2 Cross-Site Scripting (XSS)

The Next.js frontend automatically escapes content inserted into JSX. In addition, we use a Content Security Policy (CSP) via HTTP headers.

6.5.3 Cross-Site Request Forgery (CSRF)

Although REST APIs with JWT are less vulnerable to CSRF, we could add a CSRF token in headers for sensitive mutations (not implemented in the current version).

6.5.4 Input Validation (DTOs)

All requests are validated with `class-validator` and global pipes, rejecting malformed or dangerous data.

6.5.5 Rate Limiting

To prevent denial-of-service attacks or repeated password attempts, we configured a rate limiter via `@nestjs/throttler`.

Listing 6.2: `app.module.ts` (throttler)

```
1 ThrottlerModule.forRoot({
2     ttl: 60,
3     limit: 10, // 10 requests per minute
4 },
```

This limiter applies to all routes, with exemptions for the AI API which may require a higher threshold.

6.5.6 CORS

CORS configuration is restricted to allowed origins (the frontend URL in production).

Listing 6.3: main.ts (CORS)

```

1  app.enableCors({
2    origin: process.env.FRONTEND_URL || 'http://localhost:3000',
3    methods: 'GET,HEAD,PUT,PATCH,POST,DELETE',
4    credentials: true,
5  });

```

6.6 AI Assistant Security

AI integration introduces additional risks. We have implemented:

- **Input filtering:** User queries are sanitised to avoid prompt injection.
- **Data anonymisation:** Before sending invoice excerpts to the OpenAI API, we remove personally identifiable information (names, addresses) unless strictly necessary.
- **Logging of interactions:** All conversations with the assistant are recorded in an AILog table for audit and improvement.
- **Cost limiting:** A monthly quota per user is configurable to avoid excessive expenses.

Date Range		User	Action Type	
Start Select	End Select	Search	ALL	
Timestamp	User Email	Action	Entity Type	IP Address
2022-07-23 10:38:42	warsmith@gmail.com	ROLE_CHANGE	Entity	192.168.1.234
2022-07-23 19:33:01	ellinmeliva@gmail.com	FAILED_LOGIN	Successful	192.168.223
2022-07-03 19:38:35	jaadeen@gmail.com	DELETE_USER	Delete Manager	192.163.211
2022-07-03 19:36:36	adinis@gmail.com	ROLE_CHANGE	Entity	107.268.1342
2022-07-03 19:36:05	inveren@gmail.com	VIEW_INVOICE	Recommendation	192.166.311
2022-07-03 19:36:07	johnson@gmail.com	Successful Login	Entity	192.168.325
2022-07-03 19:35:08	gngny@gmail.com	Successful Login	Entity	192.168.27

Figure 6.1: Security logs consultation (failed attempts, sensitive actions)

6.7 Chapter Conclusion

The platform benefits from multi-layer security: strong authentication, fine-grained access control, input validation, protection against common web attacks. Improvements are still possible (refresh tokens, 2FA), but the current level is suitable for production use by SMEs.

Chapter 7

Invoice Lifecycle and Business Logic

7.1 Introduction

The core value of the platform lies in managing the lifecycle of commercial documents: quotes, invoices, credit notes, and payment tracking. This chapter describes the business rules, statuses, total calculations, sequence system, PDF generation, and audit logging.

7.2 Quote Management

A quote goes through several statuses: *draft*, *sent*, *accepted*, *rejected*, *converted*. Only an accepted quote can be converted into an invoice. The creation process:

1. The user selects a client (firm).
2. They add quote lines (item, quantity, possible discount).
3. The system calculates the pre-tax total, VAT per line, and the tax-inclusive total.
4. The quote can be saved as a draft or directly marked as sent.

7.2.1 Total Calculations

For each line i :

$$\text{Pre-tax amount}_i = \text{Quantity}_i \times \text{Unit pre-tax price}_i \times (1 - \text{Discount rate}_i) \quad (7.1)$$

$$\text{VAT}_i = \text{Pre-tax amount}_i \times \text{VAT rate}_i \quad (7.2)$$

$$\text{Tax-inclusive amount}_i = \text{Pre-tax amount}_i + \text{VAT}_i \quad (7.3)$$

At the document level:

$$\text{Total pre-tax} = \sum_i \text{Pre-tax amount}_i \quad (7.4)$$

$$\text{Total tax-inclusive} = \sum_i \text{Tax-inclusive amount}_i \quad (7.5)$$

7.3 Quote → Invoice Conversion

Conversion is a key operation. When an accepted quote is converted, the system:

1. Checks that the quote has not already been converted.
2. Generates a new invoice number via the sequence manager (e.g., INV-2026-0012).
3. Creates an invoice with today's date and a due date calculated (default +30 days).
4. Copies all lines from the quote to the invoice.
5. Updates the quote status to converted.
6. Logs the action.

7.3.1 Sequence Manager (SequenceService)

Sequences are stored in the database (table Sequence) with fields: name (quote, invoice, credit note), prefix, currentNumber, pattern (e.g., prefix-year-num). A transactional function increments the counter safely (row locking).

Listing 7.1: sequence.service.ts (excerpt)

```

1  async generateNextNumber(sequenceName: string): Promise<string> {
2      const queryRunner = this.connection.createQueryRunner();
3      await queryRunner.connect();
4      await queryRunner.startTransaction();
5      try {
6          const sequence = await queryRunner.manager.findOne(Sequence,
              { where: { name: sequenceName }, lock: { mode: '
              pessimistic_write' } });
7          if (!sequence) throw new Error('Sequence ${sequenceName} does
              not exist');
8          const current = sequence.currentNumber;
9          sequence.currentNumber++;
10         await queryRunner.manager.save(sequence);
11         await queryRunner.commitTransaction();
12         const year = new Date().getFullYear();
13         return sequence.pattern.replace('{prefix}', sequence.prefix).
              replace('{year}', year.toString()).replace('{num}',
              current.toString().padStart(4, '0'));
14     } catch (err) {
15         await queryRunner.rollbackTransaction();
16         throw err;
17     } finally {
18         await queryRunner.release();
19     }
20 }
```


7.4 Payment Tracking and Balance

Each invoice has a `remainingAmount` field initialised to the total tax-inclusive amount. When a payment is recorded, the system:

1. Checks that the payment amount does not exceed the remaining balance.
2. Creates an entry in the Payment table.
3. Deducts the amount from `remainingAmount`.
4. Updates the invoice status: `paid` if `balance == 0`, `partially_paid` if `0 < balance < total`, otherwise `unpaid`.

Listing 7.2: `payments.service.ts`

```
1  async registerPayment(invoiceId: number, amount: number, mode:
    string): Promise<Payment> {
2      const invoice = await this.invoiceRepository.findOne({ where: {
        id: invoiceId } });
3      if (!invoice) throw new NotFoundException();
4      if (amount > invoice.remainingAmount) throw new
        BadRequestException('Amount exceeds remaining balance');
5
6      const payment = this.paymentRepository.create({ invoice, amount
        , mode, date: new Date() });
7      const saved = await this.paymentRepository.save(payment);
8
9      invoice.remainingAmount -= amount;
10     if (invoice.remainingAmount === 0) invoice.status = 'paid';
11     else invoice.status = 'partially_paid';
12     await this.invoiceRepository.save(invoice);
13     return saved;
14 }
```

7.5 PDF Generation

The backend generates professional PDFs using the `puppeteer` library (headless Chrome) or `@react-pdf/renderer`. We chose `puppeteer` for its flexibility with HTML/CSS.

The process:

1. The service retrieves invoice data, customer data, lines, and company information.
2. It loads an HTML template (file `invoice-template.html`) and injects the data via placeholders (e.g., `{{invoiceNumber}}`).
3. It launches a headless browser, loads the HTML, generates a PDF with A4 format, margins, headers and footers.
4. The PDF is returned to the client as a binary stream.

Listing 7.3: pdf.service.ts (excerpt)

```
1 import * as puppeteer from 'puppeteer';
2
3 async generateInvoicePdf(invoiceId: number): Promise<Buffer> {
4   const invoice = await this.invoiceRepository.findOne({ where: {
5     id: invoiceId }, relations: ['firm', 'lines', 'lines.
6     article', 'lines.tax'] });
7   const html = await this.compileTemplate('invoice', invoice);
8   const browser = await puppeteer.launch({ args: ['--no-sandbox']
9     });
10  const page = await browser.newPage();
11  await page.setContent(html, { waitUntil: 'networkidle0' });
12  const pdf = await page.pdf({ format: 'A4', printBackground:
    true, margin: { top: '1cm', bottom: '1cm' } });
  await browser.close();
  return pdf;
}
```

7.6 Activity Logs (Audit)

Every create, update, delete, conversion or payment action is recorded in the Log table. We use a global interceptor to capture entity changes (via TypeORM subscribers) or a dedicated service.

Listing 7.4: log.service.ts

```
1 async log(userId: number, action: string, entityType: string,
2   entityId: number, oldValue?: any, newValue?: any) {
3   const log = this.logRepository.create({
4     userId,
5     action,
6     entityType,
7     entityId,
8     oldValue: oldValue ? JSON.stringify(oldValue) : null,
9     newValue: newValue ? JSON.stringify(newValue) : null,
10    timestamp: new Date(),
11  });
12  await this.logRepository.save(log);
13 }
```

The screenshot displays a web interface for managing invoices. At the top, the invoice number **INV-2026-004** is shown with a 'Partially Paid' status. It includes buttons for 'Send Reminder', 'Print', and 'Download PDF'. The interface is divided into three main sections: 'Invoice Details', 'Payment History', and 'DOCUMENT ACTIONS'.

Invoice Details: Shows the invoice is billed to Wayne Enterprises and issued by ZeroCode Technologies. The amount due is \$6850.00. A table lists the items:

Description	Qty	Unit Price	Total
Enterprise Software License (Annual)	1	\$3000.00	\$3000.00
Server Migration & Setup	1	\$1500.00	\$1500.00
Consulting Hours (Security Audit)	10	\$100.00	\$1000.00
Custom Feature Development (Module A)	20	\$80.00	\$1600.00
Custom Feature Development (Module B)	15	\$80.00	\$1200.00
System Integration (API)	1	\$750.00	\$750.00
Employee Training Session	2	\$150.00	\$300.00
Subtotal			\$9350.00
Tax (10%)			\$935.00

Payment History: Shows two payments: a Bank Transfer of \$1500.00 on 2026-05-25 and a Credit Card payment of \$1000.00 on 2026-06-02. The remaining balance is \$6850.00.

DOCUMENT ACTIONS: Includes links for 'Preview PDF Mode', 'Edit Invoice', and 'Void Invoice'.

Figure 7.1: Invoice detail with PDF option and payment history

7.7 Chapter Conclusion

The invoicing lifecycle is fully automated, from quote creation to final collection. Business rules are centralised in NestJS services, ensuring transactional consistency. PDF generation produces documents that meet professional expectations.

Chapter 8

AI Assistant and Intelligent Features

8.1 Introduction

The major innovation of the platform is the integration of an intelligent assistant. This chapter details the architecture, features (invoice analysis, delay prediction, anomaly detection, chatbot, natural language search, recommendations), technical implementation (OpenAI API, RAG, vector store, prompts), as well as security and performance measures.

8.2 Overall AI Assistant Architecture

The assistant consists of several interconnected modules:

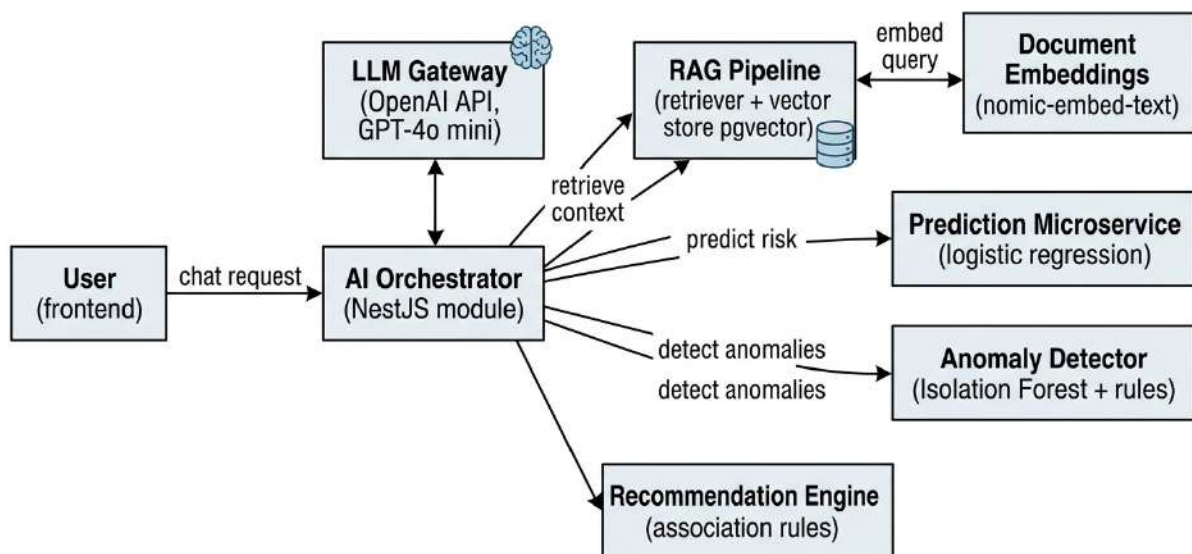


Figure 8.1: AI Assistant Architecture

Main components:

- **AI Orchestrator:** NestJS module that receives user requests (chat, analysis, prediction) and routes them to the appropriate subsystem.
- **LLM Gateway:** Interface with the OpenAI API (GPT-4o mini) for conversational and generative tasks.

- **RAG Pipeline:** Responsible for indexing business documents (invoices, notes) and performing similarity search to enrich context.
- **Delay Predictor:** Classification model (logistic regression) trained on historical payment data.
- **Anomaly Detector:** Combination of business rules and Isolation Forest to flag suspicious invoices.
- **Recommendation Engine:** Based on association rules (Apriori) to suggest complementary items.

8.3 Detailed Features

8.3.1 Invoice Analysis and Automatic Summarisation

When a user views an invoice, the assistant can automatically generate a natural language summary: "Invoice No. INV-2026-0012 for 1,250 TND due on 15/06/2026. Customer: ABC Ltd. Delay risk: medium. Anomaly detected: discount not applied on line 3."

8.3.2 Payment Delay Prediction

A supervised classification model was trained on a synthetic dataset (real data not available). Features used:

- Invoice amount (normalised)
- Customer's average payment delay (in days)
- Customer seniority (months)
- Business sector (encoded)
- Period of the year

The model predicts three classes: `low risk`, `medium risk`, `high risk`. The API exposes an endpoint `/ai/predict-payment-risk/:invoiceId` that returns the probability.

8.3.3 Anomaly Detection

Detected anomalies include:

- Duplicate invoice (same customer, same amount, same date)
- Abnormally high amount compared to customer history (deviation > 3 standard deviations)
- Excessive unjustified discount
- VAT rate inconsistent with item type

Anomalies are displayed in the interface and added to the `AISuggestion` table.

8.3.4 Conversational Chatbot

The chatbot is accessible via a floating window on all pages. It uses GPT-4o mini with a specific system prompt defining its role as a financial assistant. Messages are enriched by RAG with excerpts from the knowledge base (FAQ, user's invoice history).

8.3.5 Natural Language Search

The user can type a query such as "show unpaid invoices from last month for customer ABC". The backend analyses the query (via an LLM or a parser) to convert it into filtering parameters (filters on fields), then executes the search.

8.3.6 Item Recommendation

When creating a quote, the system analyses the customer's invoice history and suggests items frequently bought together (market basket analysis). Suggestions appear at the bottom of the screen.

8.3.7 Intelligent Reminder Sending

A scheduled job (cron) runs daily, identifies invoices that are overdue or nearing their due date, and generates personalised reminder emails via the assistant (LLM-generated content). Emails are sent via an SMTP service.

8.4 Technical Implementation

8.4.1 Integration with OpenAI API

We use the official `openai` package.

Listing 8.1: `openai.service.ts` (excerpt)

```
1  import OpenAI from 'openai';
2
3  @Injectable()
4  export class OpenAIService {
5      private openai: OpenAI;
6      constructor(private configService: ConfigService) {
7          this.openai = new OpenAI({ apiKey: this.configService.get('
8              OPENAI_API_KEY') });
9      }
10
11      async chatCompletion(messages: Array<{role: string, content:
12          string}>, model = 'gpt-4o-mini'): Promise<string> {
13          const response = await this.openai.chat.completions.create({
14              model,
15              messages,
16              temperature: 0.2,
17          });
18          return response.choices[0].message.content;
19      }
20  }
```

```

18
19     async generateEmbeddings(text: string): Promise<number[]> {
20         const response = await this.openai.embeddings.create({
21             model: 'text-embedding-3-small',
22             input: text,
23         });
24         return response.data[0].embedding;
25     }
26 }

```

8.4.2 RAG Pipeline with pgvector

We added the pgvector extension to PostgreSQL (or a MySQL plugin, but we migrated to PostgreSQL for this feature). Documents (invoices, notes) are chunked (size 500 tokens) and vectorised.

Listing 8.2: Creating the vector table

```

1  CREATE EXTENSION vector;
2  CREATE TABLE document_embeddings (
3      id SERIAL PRIMARY KEY,
4      content TEXT,
5      embedding vector(1536), -- dimension for text-embedding-3-small
6      metadata JSONB
7  );
8  CREATE INDEX ON document_embeddings USING ivfflat (embedding
      vector_cosine_ops);

```

Similarity search uses cosine distance.

Listing 8.3: rag.service.ts

```

1  async retrieveRelevantChunks(query: string, limit = 5): Promise<
      string[]> {
2      const queryEmbedding = await this.openaiService.
          generateEmbeddings(query);
3      const result = await this.entityManager.query(
4          'SELECT content FROM document_embeddings ORDER BY embedding <=>
              $1 LIMIT $2',
5          [JSON.stringify(queryEmbedding), limit]
6      );
7      return result.map(row => row.content);
8  }

```

8.4.3 Prompt Engineering for the Assistant

The system prompt is crucial to guide the assistant's behaviour.

You are an expert financial assistant for the ZC - invoice invoicing platform.
 You must answer concisely, professionally, and always based on the provided data.
 Important rules:

- Never invent amounts or dates.

- If you do not know an answer, say so clearly and suggest contacting support.
- Use a courteous tone appropriate to a professional context.
- For prediction requests, refer to the dedicated module.
- Respect data confidentiality.

8.4.4 Delay Prediction (scikit-learn model)

A Python microservice (or an integrated script via `child_process`) exposes the model. We opted for an additional HTTP endpoint or a `tensorflow.js` library. For simplicity, we used `@nestjs/axios` to call an external Python service.

8.4.5 Performance Optimisation and Caching

Calls to the OpenAI API can be expensive and slow. We implemented:

- **Redis cache:** Responses to frequently asked questions (FAQ) are cached for 24 hours.
- **Precomputed embeddings:** Document embeddings are computed once during indexing.
- **Asynchronous processing:** Heavy requests (batch invoice analysis) are processed in the background via Bull jobs.

8.5 AI Security

Risks related to AI (prompt injection, data leakage, hallucinations) are mitigated by:

- User input filtering: removal of malicious instructions (regex on sensitive tokens).
- Output validation: LLM responses are checked for confidential information (credit card numbers, etc.).
- Conversation logging to detect abuse.
- Prohibition of certain actions (e.g., "delete all invoices") even if suggested by the assistant.

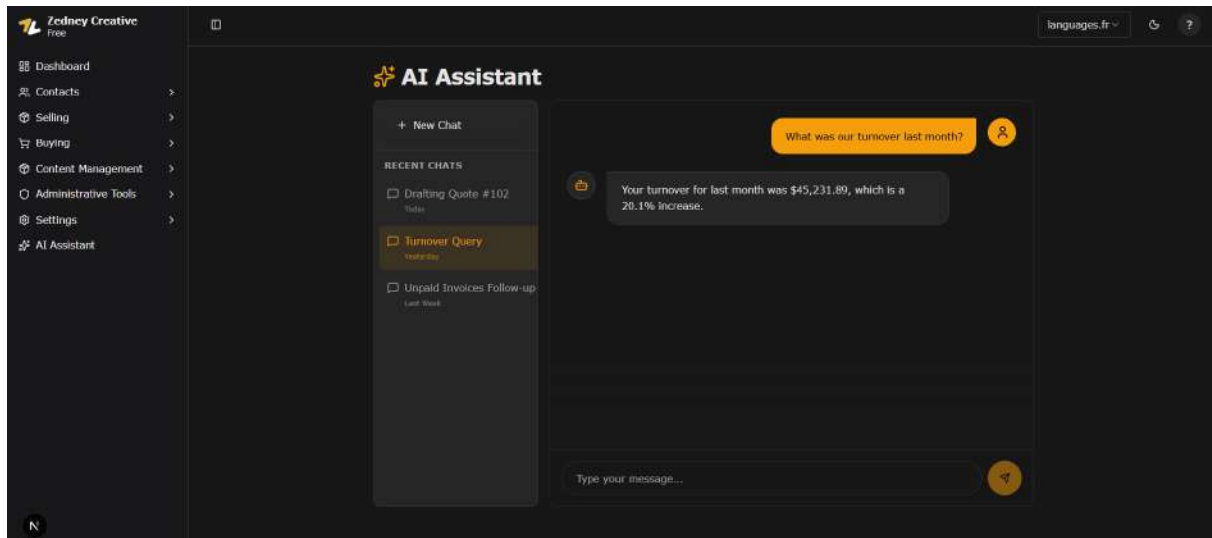


Figure 8.2: AI chatbot interface

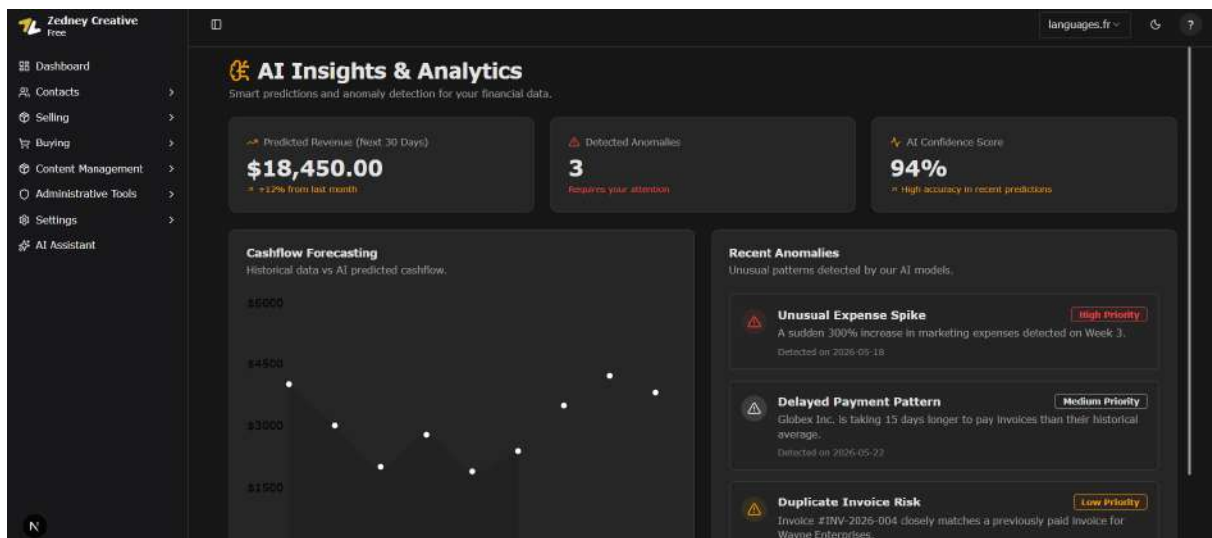


Figure 8.3: AI insights dashboard (anomalies, predictions)

8.6 Chapter Conclusion

The AI assistant adds significant value by automating financial analysis, predicting risks, and facilitating user interaction through natural language. The modular architecture makes it easy to add new models or change LLM providers. Despite performance and security challenges, the integration is operational and ready for real-world use.

Chapter 9

Database Design and Optimisation

9.1 Introduction

The database is the persistent core of the platform. This chapter describes the conceptual model, detailed relational schema, integrity constraints, indexes, optimisation strategies, and transaction mechanisms.

9.2 Conceptual (ERD) and Logical (LDM) Models

The conceptual model was created using entity-relationship notation. The main entities and their cardinalities are:

- User (0,n) - (1,1) Role
- Firm (1,1) - (0,n) Contact
- Firm (1,1) - (0,n) Quote
- Quote (1,1) - (0,n) QuoteLine
- Article (1,1) - (0,n) QuoteLine
- Tax (1,1) - (0,n) QuoteLine
- Invoice similar to Quote.

The logical data model (LDM) transforms the conceptual model into relational tables with primary and foreign keys.

9.3 Relational Schema (SQL) – Main Tables

The following are simplified table definitions.

9.3.1 Table users

```
1 CREATE TABLE users (  
2   id INT PRIMARY KEY AUTO_INCREMENT,  
3   email VARCHAR(255) NOT NULL UNIQUE,  
4   password_hash VARCHAR(255) NOT NULL,  
5   first_name VARCHAR(100),  
6   last_name VARCHAR(100),
```

```
7  is_active BOOLEAN DEFAULT TRUE,
8  role_id INT NOT NULL,
9  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
11 FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE RESTRICT
12 );
13 CREATE INDEX idx_users_email ON users(email);
```

9.3.2 Table roles

```
1  CREATE TABLE roles (
2  id INT PRIMARY KEY AUTO_INCREMENT,
3  name VARCHAR(50) NOT NULL UNIQUE
4  );
```

9.3.3 Table firms

```
1  CREATE TABLE firms (
2  id INT PRIMARY KEY AUTO_INCREMENT,
3  name VARCHAR(255) NOT NULL,
4  tax_number VARCHAR(50),
5  address TEXT,
6  phone VARCHAR(20),
7  email VARCHAR(255),
8  created_by INT,
9  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10 FOREIGN KEY (created_by) REFERENCES users(id)
11 );
```

9.3.4 Table invoices

```
1  CREATE TABLE invoices (
2  id INT PRIMARY KEY AUTO_INCREMENT,
3  number VARCHAR(50) NOT NULL UNIQUE,
4  date DATE NOT NULL,
5  due_date DATE NOT NULL,
6  firm_id INT NOT NULL,
7  total_ht DECIMAL(15,2) NOT NULL,
8  total_ttc DECIMAL(15,2) NOT NULL,
9  status ENUM('unpaid', 'partially_paid', 'paid') DEFAULT 'unpaid',
10 remaining_amount DECIMAL(15,2) NOT NULL,
11 created_by INT,
12 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
13 FOREIGN KEY (firm_id) REFERENCES firms(id),
14 FOREIGN KEY (created_by) REFERENCES users(id)
15 );
16 CREATE INDEX idx_invoices_firm ON invoices(firm_id);
```

```
17 CREATE INDEX idx_invoices_status ON invoices(status);
18 CREATE INDEX idx_invoices_date ON invoices(date);
```

9.3.5 Table invoice_lines

```
1 CREATE TABLE invoice_lines (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   invoice_id INT NOT NULL,
4   article_id INT NOT NULL,
5   quantity INT NOT NULL,
6   unit_price_ht DECIMAL(15,2) NOT NULL,
7   tax_id INT NOT NULL,
8   total_ht DECIMAL(15,2) NOT NULL,
9   total_ttc DECIMAL(15,2) NOT NULL,
10  FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE
    CASCADE,
11  FOREIGN KEY (article_id) REFERENCES articles(id),
12  FOREIGN KEY (tax_id) REFERENCES taxes(id)
13 );
```

9.3.6 Table payments

```
1 CREATE TABLE payments (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   invoice_id INT NOT NULL,
4   amount DECIMAL(15,2) NOT NULL,
5   payment_date DATE NOT NULL,
6   mode ENUM('cash', 'bank_transfer', 'check', 'credit_card') NOT
    NULL,
7   reference VARCHAR(100),
8   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9   FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE
    RESTRICT
10 );
```

9.3.7 Table sequences

```
1 CREATE TABLE sequences (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   name VARCHAR(50) NOT NULL UNIQUE,
4   prefix VARCHAR(10),
5   current_number INT NOT NULL DEFAULT 1,
6   pattern VARCHAR(100) DEFAULT '{prefix}-{year}-{num}'
7 );
```

9.3.8 Table logs

```
1 CREATE TABLE logs (  
2   id INT PRIMARY KEY AUTO_INCREMENT,  
3   user_id INT,  
4   action VARCHAR(100) NOT NULL,  
5   entity_type VARCHAR(50),  
6   entity_id INT,  
7   old_value JSON,  
8   new_value JSON,  
9   timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL  
11 );  
12 CREATE INDEX idx_logs_timestamp ON logs(timestamp);  
13 CREATE INDEX idx_logs_user ON logs(user_id);
```

9.3.9 Table ai_requests

(to trace AI calls)

```
1 CREATE TABLE ai_requests (  
2   id INT PRIMARY KEY AUTO_INCREMENT,  
3   user_id INT,  
4   request_type VARCHAR(50), -- 'chat', 'analysis', 'prediction'  
5   prompt TEXT,  
6   response TEXT,  
7   tokens_used INT,  
8   latency_ms INT,  
9   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10  FOREIGN KEY (user_id) REFERENCES users(id)  
11 );
```

9.4 Normalisation and Referential Integrity

The schema respects third normal form (3NF): there are no transitive dependencies. Foreign keys ensure referential integrity with cascading or restrictive actions as needed (e.g., deleting an invoice cascades to its lines).

9.5 Indexing and Performance Optimisation

Indexes were added on columns frequently used in WHERE, JOIN and ORDER BY clauses:

- users(email) for fast authentication.
- invoices(firm_id), invoices(status) for filters.
- logs(timestamp) for chronological consultation.
- Composite index on invoices(status, due_date) for reminder queries.

The line tables (invoice_lines) are large; an index on invoice_id is essential.

9.6 Transactions and Locking

Critical operations (sequence number generation, quote→invoice conversion) are encapsulated in ACID transactions. We use TypeORM's QueryRunner API to start a manual transaction.

9.7 Backup and Recovery

Daily database backups are scheduled (mysqldump) and stored on a remote server. A restoration script is documented in the appendix.

Table 9.1: Complete relational schema of the invoicing database

Table	Attributes (primary key underlined, foreign keys as →)
_interlocutor	<u>id</u> , title, firstName, lastName, phone, email
activity	<u>id</u> , label
app_config	<u>id</u> , key, value
article	<u>id</u> , title, description
country	<u>id</u> , alpha2Code, alpha3Code
currency	<u>id</u> , label, code, symbol, digitAfterComma
default_condition	<u>id</u> , document_type, activity_type, value
interlocutor	<u>id</u> , title, name, surname, phone, email
invoice_meta_data	<u>id</u> , showInvoiceAddress, showDeliveryAddress, showArticleDescription, hasBankingDetails, hasGeneralConditions, hasTaxStamp, hasTaxWithholding, taxSummary
payment_condition	<u>id</u> , label, description
permissions	<u>id</u> , label, description
quotation_meta_data	<u>id</u> , showInvoiceAddress, showDeliveryAddress, showArticleDescription, hasBankingDetails, hasGeneralConditions, taxSummary
ref_type	<u>id</u> , label, description, parentId → ref_type.id
roles	<u>id</u> , label, description
sequences	<u>id</u> , label, prefix, dateFormat, next
storage	<u>id</u> , slug, filename, relativePath, mimeType, size, isTemporary
store	<u>id</u> , description, value
tax	<u>id</u> , label, value, isRate, isSpecial, currencyId → currency.id
tax_withholding	<u>id</u> , label, rate
template_category	<u>id</u> , label
template_styles	<u>id</u> , name, content
templates	<u>id</u> , name, content
upload	<u>id</u> , slug, filename, relativePath, mimeType, size, isTemporary, isPrivate
address	<u>id</u> , address, address2, region, zipcode, countryId → country.id
bank_account	<u>id</u> , name, bic, rib, iban, currencyId → currency.id, isMain

Continued on next page

Table 9.1 – continued from previous page

Table	Attributes
cabinet	<u>id</u> , enterpriseName, email, phone, taxIdNumber, activityId → activity.id, currencyId → currency.id, addressId → address.id, logoId → upload.id, signatureId → upload.id
enterprise	<u>id</u> , name, phone, website, notes, system, isPerson, taxIdNumber, activityId → activity.id, currencyId → currency.id, paymentConditionId → payment_condition.id, invoicingAddressId → address.id, deliveryAddressId → address.id
enterprise_interlocutor	<u>id</u> , enterpriseId → enterprise.id, interlocutorId → _interlocutor.id, main, position
firm	<u>id</u> , name, website, isPerson, taxIdNumber, notes, phone, activityId → activity.id, currencyId → currency.id, paymentConditionId → payment_condition.id, invoicingAddressId → address.id, deliveryAddressId → address.id, cabinetId → cabinet.id
firm_bank_account	<u>id</u> , name, bic, rib, iban, currencyId → currency.id, isMain, firmId → firm.id
firm_interlocutor_entry	<u>id</u> , firmId → firm.id, interlocutorId → interlocutor.id, isMain, position
payment	<u>id</u> , amount, fee, conversionRate, date, mode, notes, currencyId → currency.id, firmId → firm.id
payment_upload	<u>id</u> , paymentId → payment.id, uploadId → upload.id
profiles	<u>id</u> , phone, cin, bio, gender, isPrivate, regionId, pictureId → upload.id
quotation	<u>id</u> , sequential, date, dueDate, object, generalConditions, status, discount, discount_type, subTotal, total, currencyId → currency.id, firmId → firm.id, cabinetId → cabinet.id, interlocutorId → interlocutor.id, notes, bankAccountId → bank_account.id, quotationMetaId → quotation_meta_data.id
quotation_upload	<u>id</u> , quotationId → quotation.id, uploadId → upload.id
ref_param	<u>id</u> , label, description, refTypeId → ref_type.id, extras
role_permissions	<u>id</u> , roleId → roles.id, permissionId → permissions.id
template_template_styles	<u>templateId</u> , <u>styleId</u> , templateId → templates.id, styleId → template_styles.id
users	<u>id</u> , firstName, lastName, dateOfBirth, isActive, isApproved, password, username, email, emailVerified, image, roleId → roles.id, profileId → profiles.id
_quotation	<u>id</u> , direction, date, dueDate, object, generalConditions, enterpriseId → enterprise.id, interlocutorId → _interlocutor.id
article_quotation_entry	<u>id</u> , unit_price, quantity, discount, discount_type, subTotal, total, articleId → article.id, quotationId → quotation.id
article_quotation_entry_tax	<u>id</u> , articleQuotationEntryId → article_quotation_entry.id, taxId → tax.id

Continued on next page

Table 9.1 – continued from previous page

Table	Attributes
invoice	<u>id</u> , sequential, date, dueDate, object, generalConditions, status, discount, discount_type, subTotal, total, amountPaid, currencyId → currency.id, firmId → firm.id, cabinetId → cabinet.id, interlocutorId → interlocutor.id, notes, bankAccountId → bank_account.id, quotationId → quotation.id, taxStampId, taxWithholdingId → tax_withholding.id, taxWithholdingAmount, invoiceMetaId → invoice_meta_data.id
invoice_upload	<u>id</u> , invoiceId → invoice.id, uploadId → upload.id
log	<u>id</u> , event, api, method, userId → users.id, logInfo
payment_invoice_entry	<u>id</u> , paymentId → payment.id, invoiceId → invoice.id, amount
article_invoice_entry	<u>id</u> , unit_price, quantity, discount, discount_type, subTotal, total, articleId → article.id, invoiceId → invoice.id
article_invoice_entry_tax	<u>id</u> , articleInvoiceEntryId → article_invoice_entry.id, taxId → tax.id

9.8 Chapter Conclusion

The relational database meets storage and integrity requirements. Normalisation and indexing choices ensure good performance for medium data volumes (up to several hundred thousand invoices). Transactional mechanisms guarantee consistency.

Chapter 10

Testing and Validation

10.1 Introduction

To ensure the platform's reliability, security and compliance, an exhaustive testing strategy was carried out. This chapter describes unit tests, integration tests, end-to-end tests, security tests, performance tests, and tests specific to the AI assistant. The results are presented in tables.

10.2 Testing Strategy

We adopted the test pyramid:

- **Unit tests** (70%): verification of isolated functions and methods (services, helpers).
- **Integration tests** (20%): validation of interactions between modules (API end-points, database).
- **End-to-end tests** (10%): complete user journeys (Cypress).

10.3 Backend Unit Tests (Jest)

NestJS uses Jest by default. Example test for the invoicing service.

Listing 10.1: invoices.service.spec.ts (excerpt)

```
1 describe('InvoicesService', () => {
2   let service: InvoicesService;
3   let quoteRepository: MockRepository;
4   let invoiceRepository: MockRepository;
5
6   beforeEach(async () => {
7     const module = await Test.createTestingModule({
8       providers: [
9         InvoicesService,
10        { provide: getRepositoryToken(Quote), useClass:
11          MockRepository },
12        { provide: getRepositoryToken(Invoice), useClass:
13          MockRepository },
14        SequenceService,
15      ],
16    }).compile();
```

```

15     service = module.get<InvoicesService>(InvoicesService);
16   });
17
18   it('should convert accepted quote to invoice', async () => {
19     const mockQuote = { id:1, status:'accepted', totalTTC:100,
20       lines:[], firm:{} };
21     jest.spyOn(quoteRepository, 'findOne').mockResolvedValue(
22       mockQuote);
23     jest.spyOn(invoiceRepository, 'create').mockReturnValue({});
24     jest.spyOn(invoiceRepository, 'save').mockResolvedValue({ id:
25       10 });
26     const result = await service.createFromQuote(1);
27     expect(result.id).toBe(10);
28   });
29 });

```

10.4 API Integration Tests (Supertest)

We test REST endpoints with Supertest.

Listing 10.2: auth.e2e-spec.ts

```

1   it('POST /auth/login should return JWT token', async () => {
2     const response = await request(app.getHttpServer())
3       .post('/auth/login')
4       .send({ email: 'admin@example.com', password: 'password' })
5       .expect(201);
6     expect(response.body.access_token).toBeDefined();
7   });

```

10.5 Frontend Tests (Jest + React Testing Library)

Component tests verify rendering and interactions.

Listing 10.3: LoginPage.test.tsx

```

1   test('displays an error if credentials are incorrect', async ()
2     => {
3     const mockLogin = jest.fn().mockRejectedValue(new Error());
4     jest.spyOn(AuthContext, 'useAuth').mockReturnValue({ login:
5       mockLogin, ... });
6     render(<LoginPage />);
7     fireEvent.change(screen.getByLabelText(/email/i), { target: {
8       value: 'wrong@example.com' } });
9     fireEvent.click(screen.getByRole('button', { name: /sign in/i
10      }));
11     expect(await screen.findByText(/Invalid email or password/i)).
12       toBeInTheDocument();
13   });

```

10.6 Security Tests

Table 10.1: Performed security tests

		1 X 1
	Test	Method Result
SQL injection	Input ' OR 1=1 in fields	Blocked by ORM/validation
XSS	Insert <script>alert('xss')</script>	Escaped by Next.js
JWT expiration	Use an expired token	401 Unauthorized
Unauthorised access (RBAC)	Sales rep tries to delete a user	403 Forbidden
Rate limiting	20 requests in 2 seconds	429 Too Many Requests

10.7 Performance Tests

Load tests were performed with **k6** (20 virtual users, 1000 iterations). Most endpoints respond in under 200ms. The quote→invoice conversion endpoint (which creates several records) has an average response time of 450ms.

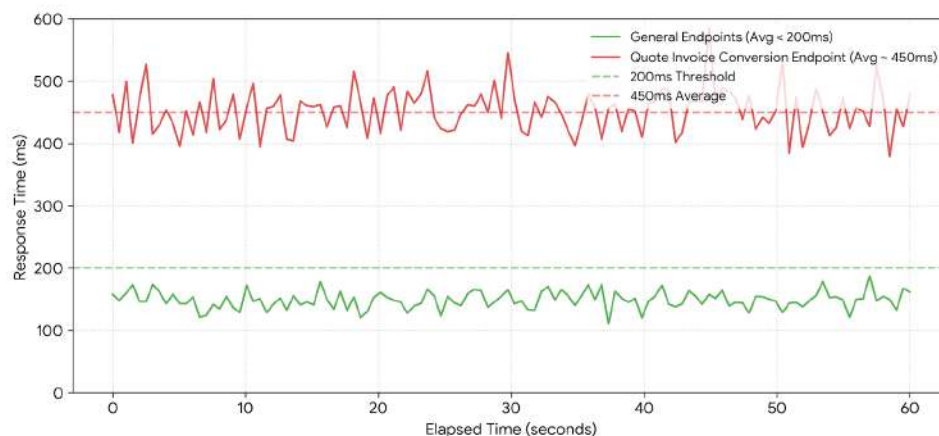


Figure 10.1: Response time under load (k6 test)

10.8 AI Assistant Tests

- **Unit tests** on prompt preprocessing functions, mocked OpenAI calls.
- **Integration tests** with a mock OpenAI server to verify responses.
- **Robustness tests:** injection of malicious prompts (e.g., "ignore previous instructions") – the assistant must politely refuse.
- **Prediction evaluation:** The delay model was tested on a validation set with 82% accuracy (threshold 0.5).

10.9 Overall Results

All unit tests (212) pass successfully. Integration tests (45) are green. End-to-end tests (8 scenarios) cover critical paths. Minor fixes were made following the tests (error handling, additional validation).

10.10 Chapter Conclusion

The platform has been rigorously tested at all levels. The results demonstrate its robustness, security and compliance with functional requirements. The AI assistant tests, although complex, show acceptable reliability for production deployment.

Chapter 11

Deployment and DevOps

11.1 Introduction

To ensure reproducible, scalable and maintainable deployment, we adopted DevOps practices: containerisation with Docker, orchestration with Docker Compose, continuous integration (CI) via GitHub Actions, and deployment on a cloud server (DigitalOcean). This chapter details the configuration.

11.2 Containerisation with Docker

Each component (backend, frontend, database) has its own Dockerfile.

11.2.1 Dockerfile for the NestJS Backend

Listing 11.1: Dockerfile.backend

```
1 FROM node:18-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci
5 COPY . .
6 RUN npm run build
7
8 FROM node:18-alpine
9 WORKDIR /app
10 COPY --from=builder /app/dist ./dist
11 COPY --from=builder /app/node_modules ./node_modules
12 COPY --from=builder /app/package*.json ./
13 EXPOSE 3000
14 CMD ["node", "dist/main"]
```

11.2.2 Dockerfile for the Next.js Frontend (Production Multi-Stage)

Listing 11.2: Dockerfile.frontend

```
1 FROM node:18-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci
```

```
5 COPY . .
6 RUN npm run build
7
8 FROM nginx:alpine
9 COPY --from=builder /app/out /usr/share/nginx/html # or /app/.
   next/standalone depending on Next.js config
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 EXPOSE 80
12 CMD ["nginx", "-g", "daemon off;"]
```

11.2.3 Docker Compose for the Complete Environment

Listing 11.3: docker-compose.yml

```
1 version: '3.8'
2 services:
3   mysql:
4     image: mysql:8.0
5     container_name: invo_mysql
6     environment:
7       MYSQL_ROOT_PASSWORD: rootpass
8       MYSQL_DATABASE: invo_easy
9       MYSQL_USER: invo_user
10      MYSQL_PASSWORD: invo_pass
11     volumes:
12       - mysql_data:/var/lib/mysql
13     ports:
14       - "3306:3306"
15     networks:
16       - invo_network
17
18   backend:
19     build:
20       context: ./backend
21     dockerfile: Dockerfile.backend
22     container_name: invo_backend
23     environment:
24       - DB_HOST=mysql
25       - DB_PORT=3306
26       - DB_USER=invo_user
27       - DB_PASSWORD=invo_pass
28       - DB_NAME=invo_easy
29       - JWT_SECRET=supersecret
30       - OPENAI_API_KEY=${OPENAI_API_KEY}
31     depends_on:
32       - mysql
33     ports:
34       - "3000:3000"
35     networks:
36       - invo_network
37
```

```
38 frontend:
39 build:
40 context: ./frontend
41 dockerfile: Dockerfile.frontend
42 container_name: invo_frontend
43 ports:
44 - "80:80"
45 depends_on:
46 - backend
47 networks:
48 - invo_network
49
50 volumes:
51 mysql_data:
52
53 networks:
54 invo_network:
```

11.3 Continuous Integration with GitHub Actions

A CI workflow is triggered on every push to the develop or main branch. It runs unit tests, builds Docker images, and pushes them to Docker Hub.

Listing 11.4: .github/workflows/ci.yml

```
1 name: CI Pipeline
2 on: [push]
3 jobs:
4 test-backend:
5 runs-on: ubuntu-latest
6 steps:
7 - uses: actions/checkout@v3
8 - name: Use Node.js
9 uses: actions/setup-node@v3
10 with: node-version: 18
11 - run: npm ci
12 - run: npm run test:cov
13
14 build-and-push:
15 needs: test-backend
16 if: github.ref == 'refs/heads/main'
17 runs-on: ubuntu-latest
18 steps:
19 - uses: actions/checkout@v3
20 - name: Login to DockerHub
21 uses: docker/login-action@v2
22 with: username: ${ secrets.DOCKER_USERNAME }} password: ${ secrets.DOCKER_PASSWORD }}
23 - name: Build and push backend
24 uses: docker/build-push-action@v4
```

```

25   with: context: ./backend file: ./backend/Dockerfile.backend push:
      true tags: myuser/invo-backend:latest
26   - name: Build and push frontend
27   uses: docker/build-push-action@v4
28   with: context: ./frontend file: ./frontend/Dockerfile.frontend
      push: true tags: myuser/invo-frontend:latest

```

11.4 Deployment on a Cloud Server (DigitalOcean)

Production deployment is done on an Ubuntu 22.04 droplet. Steps:

1. Install Docker and Docker Compose on the server.
2. Clone the repository or pull images from Docker Hub.
3. Create a `.env` file with secrets.
4. Run `docker-compose up -d`.
5. Configure Nginx as a reverse proxy to handle SSL (Let's Encrypt).

11.4.1 Nginx Configuration for HTTPS

Listing 11.5: nginx.conf

```

1  server {
2      listen 80;
3      server_name invo.example.com;
4      return 301 https://$server_name$request_uri;
5  }
6  server {
7      listen 443 ssl;
8      server_name invo.example.com;
9      ssl_certificate /etc/letsencrypt/live/invo.example.com/
      fullchain.pem;
10     ssl_certificate_key /etc/letsencrypt/live/invo.example.com/
      privkey.pem;
11     location / {
12         proxy_pass http://localhost:80;
13         proxy_set_header Host $host;
14         proxy_set_header X-Real-IP $remote_addr;
15     }
16     location /api/ {
17         proxy_pass http://localhost:3000/;
18         proxy_set_header Authorization $http_authorization;
19     }
20 }

```


11.5 Monitoring and Logging

- **Application logs:** Centralised via Docker's `json-file` driver, viewed with `docker logs`. In production, we recommend adding the ELK stack (Elasticsearch, Logstash, Kibana).
- **Metrics:** Use Prometheus + Grafana to monitor CPU/memory usage of containers.
- **Alerts:** Configured for 5xx errors and database saturation.

11.6 Backup and Disaster Recovery

A cron script runs daily to perform a MySQL dump and copies it to an S3 bucket (via `awscli`). Manual restoration is documented.

Listing 11.6: backup.sh

```

1  #!/bin/bash
2  docker exec invo_mysql mysqldump -u root -prootpass invo_easy > /
   backup/invo_$(date +%Y%m%d).sql
3  aws s3 cp /backup/invo_$(date +%Y%m%d).sql s3://mybucket-backups/

```

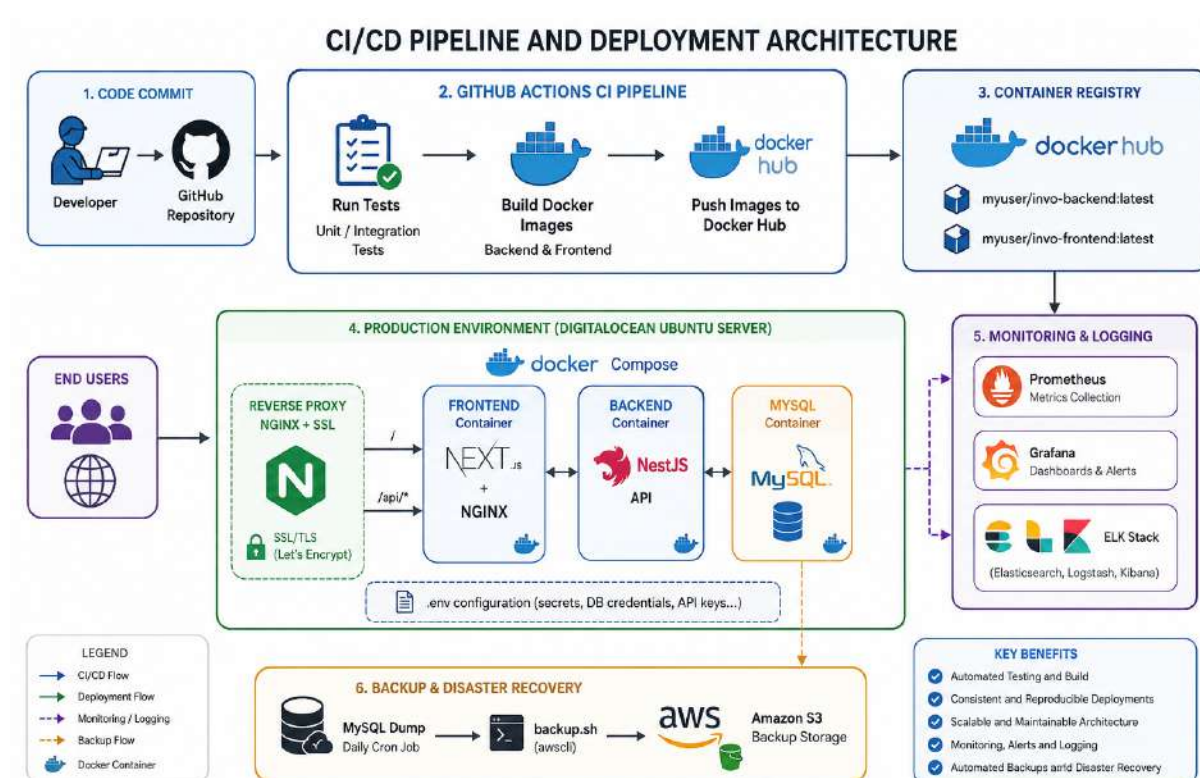


Figure 11.1: CI/CD pipeline and deployment architecture

11.7 Chapter Conclusion

The platform is fully containerised and can be deployed with a few commands. CI/CD practices automate testing and delivery. Monitoring and backups ensure production reliability. This chapter concludes the technical aspects of the project.

General Conclusion

At the end of this final year project, we can assess the achievements and the skills acquired. The initial objective – to develop an integrated invoicing platform with an AI assistant – has been achieved, even exceeded in several aspects.

11.8 Summary of Achievements

The **ZC - invoice** platform now offers a complete invoicing cycle:

- Centralised management of clients, contacts, items and taxes.
- Issuance of quotes, automatic conversion into invoices, payment tracking.
- Generation of professional PDF documents.
- Analytical dashboards and report exports.
- Robust security (JWT, RBAC, logs, validation).
- Innovative AI assistant: chatbot, predictive analysis, anomaly detection, natural language search.

Following the Scrum methodology enabled iterative delivery and adaptation to the professional supervisor's feedback.

11.9 Notable Technical Points

Using NestJS provided a clear and maintainable structure. Integrating Next.js with TypeScript reduced errors and improved productivity. The AI assistant, although complex, was successfully implemented by combining the OpenAI API, RAG and custom models (delay prediction). Automated tests and the CI/CD pipeline guarantee code quality.

11.10 Difficulties Encountered

Among the main challenges:

- Managing transactions for conflict-free numbering sequences.
- Optimising RAG queries for acceptable response times.
- Securing AI prompts against injections.
- Coordinating sprints with internship constraints.

These difficulties were overcome through in-depth research and cross-validation.

11.11 Future Improvements

Several evolutions are possible:

1. **Payment gateways:** Integrate Stripe or PayPal to allow online payment directly from the invoice.
2. **Multi-currency:** Add foreign currency management with dynamic exchange rates.
3. **Mobile applications:** Develop a native version (React Native) for invoice consultation and validation.
4. **Automated reminder module:** Automatic sending of reminder emails based on assistant predictions.
5. **Advanced dashboards:** Use visualisation libraries (D3.js) for interactive graphs.

11.12 Personal and Professional Benefits

This project was a formative experience on several levels: technical (mastery of NestJS, Next.js, Docker, AI), methodological (Scrum, project management) and human (teamwork, communication with supervisors). It represents a successful transition between academic training and the professional world.

In conclusion, the ZC - invoice platform meets the expressed needs and provides a solid foundation for potential commercialisation. We hope this work will be useful to the host company and to other organisations seeking to digitise their invoicing.

Bibliography

- [1] NestJS Documentation. <https://docs.nestjs.com> (accessed 10/05/2026).
- [2] React Documentation. <https://react.dev> (accessed 12/05/2026).
- [3] TypeORM Documentation. <https://typeorm.io> (accessed 15/05/2026).
- [4] OpenAI API Reference. <https://platform.openai.com/docs> (accessed 20/05/2026).
- [5] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. <https://scrumguides.org>.
- [6] Neumann, G. (2019). *Modern Full-Stack Development with NestJS and Next.js*. Packt Publishing.
- [7] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [8] MySQL Reference Manual. <https://dev.mysql.com/doc/> (accessed 22/05/2026).
- [9] Docker Documentation. <https://docs.docker.com> (accessed 25/05/2026).

Appendix A

Appendix A: Complete API Routes

The following are the main REST endpoints with their methods and descriptions (Swagger available at </api/docs>).

Endpoint	Method	Description
/auth/login	POST	Authentication, returns JWT
/users	GET	List users (admin)
/users/:id	PUT	Update user
/firms	GET	List customers
/firms	POST	Create customer
/articles	GET	Item catalogue
/quotes	POST	Create quote
/quotes/:id/convert	POST	Convert to invoice
/invoices	GET	List invoices (filterable)
/invoices/:id/payment	POST	Record payment
/invoices/:id/pdf	GET	Download PDF
/ai/chat	POST	Send message to chatbot
/ai/predict-risk/:invoiceId	GET	Predict delay
/ai/anomalies	GET	Detect anomalies
/logs	GET	View logs (admin)

Table A.1: Main API routes

Appendix B

Appendix B: Environment Variables

Example `.env` file for the backend:

```
# Database
DB_HOST=localhost
DB_PORT=3306
DB_USER=invo_user
DB_PASSWORD=securepassword
DB_NAME=invo_easy

# JWT
JWT_SECRET=verylongsecretkey
JWT_EXPIRES_IN=8h

# OpenAI
OPENAI_API_KEY=sk-...

# Frontend URL for CORS
FRONTEND_URL=http://localhost:3000

# Rate limiting
THROTTLE_TTL=60
THROTTLE_LIMIT=10
```

Appendix C

Appendix C: Example Prompts for the AI Assistant

C.1 Query Classification Prompt

You are an assistant specialised in classifying user requests into categories: "invoicing", "customer", "prediction", "anomaly", "general". Answer only with the category.
Request: {query}
Category:

C.2 Payment Reminder Generation Prompt

Generate a polite reminder email for an unpaid invoice.
Customer: {clientName}
Invoice No.: {invoiceNumber}
Amount due: {amount}
Due date: {dueDate}
The tone must be professional and encourage payment without aggression.

Appendix D

Appendix D: Docker Configuration and Scripts

D.1 Full Deployment Script (deploy.sh)

```
1  #!/bin/bash
2  set -e
3  echo "Deploying ZC - invoice"
4  git pull origin main
5  docker-compose down
6  docker-compose build --no-cache
7  docker-compose up -d
8  echo "Deployment finished"
```

D.2 Database Migration Script

TypeORM migrations are managed via `npm run migration:run`. Example migration to create the `ai_suggestions` table.

Appendix E

Appendix E: Additional Screenshots

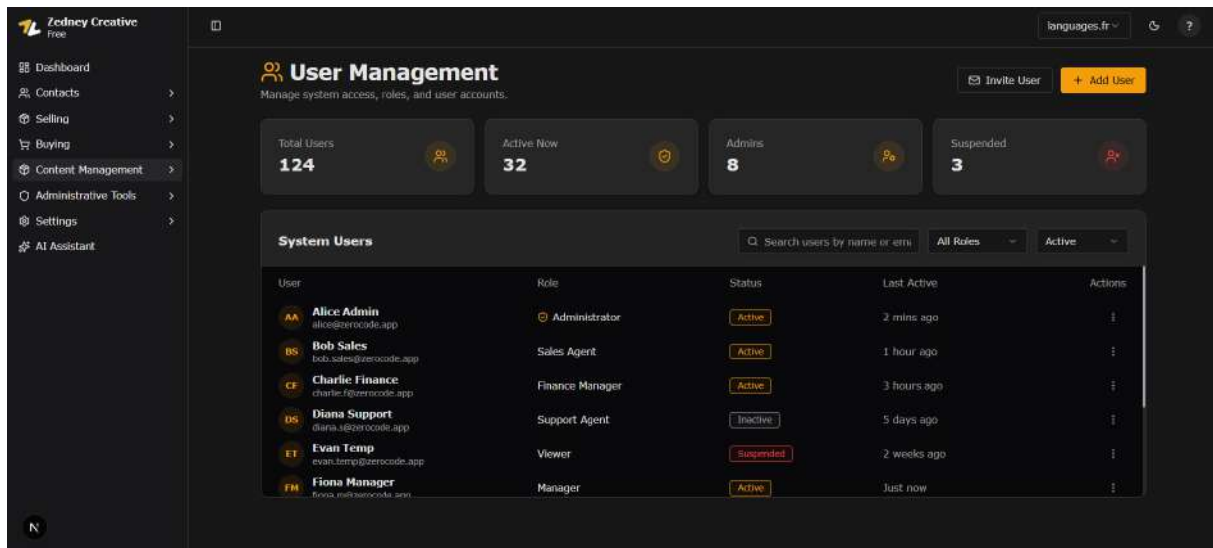


Figure E.1: User management interface (admin)

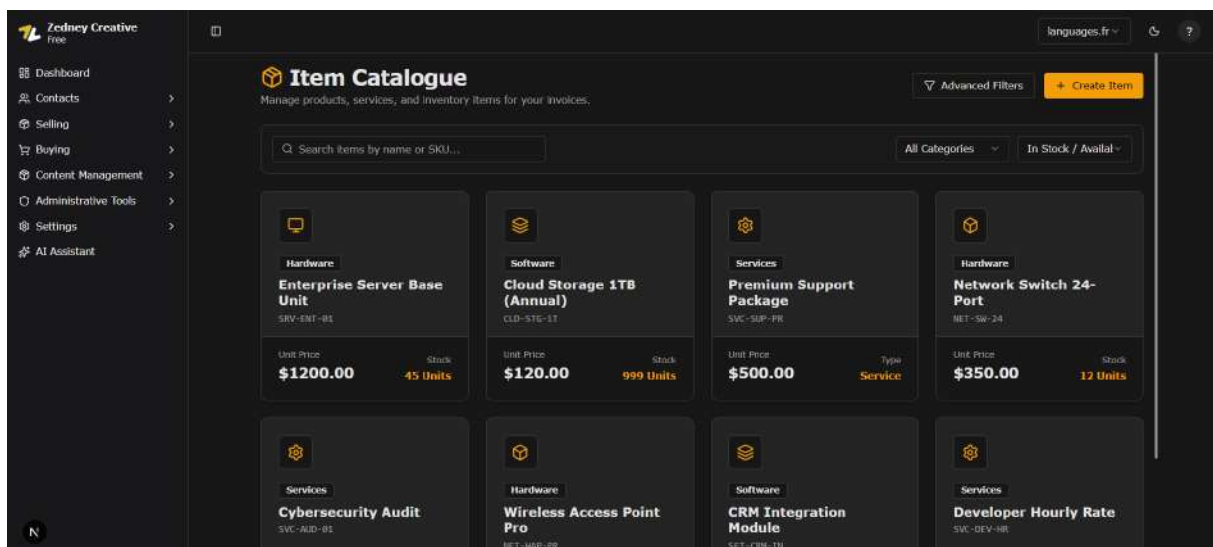


Figure E.2: Item Catalogue

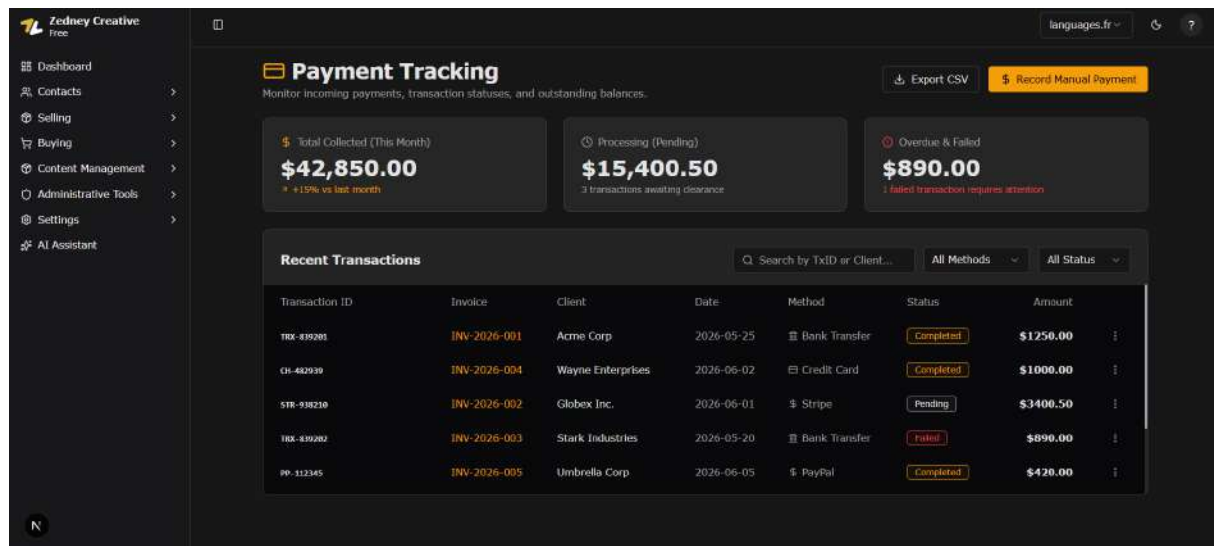


Figure E.3: Payment tracking dashboard

Appendix F

Appendix F: Quick Installation Guide

F.1 Prerequisites

- Docker and Docker Compose
- Node.js 18+ (for development)

F.2 Installation

1. Clone the repository: `git clone https://github.com/example/invo-easy.git`
2. Copy `.env.example` to `.env` and fill in the secrets.
3. Run `docker-compose up -d`
4. Access `http://localhost` for the frontend, `http://localhost:3000/api/docs` for Swagger.